

Computer Vision 1

Single Shot Detection Network Architectures

Dr. Martin R. Oswald



UNIVERSITY
OF AMSTERDAM



computer
vision lab

Lecture overview

Week		Date	Presenter	Lectures	Excerise / Labs
1	36	02/09/2024	Arnoud Visser	1. Introduction and Course Logistics; Camera model	
1	36	05/09/2024	Arnoud Visser	2. Sensor and image formation Color	
2	37	09/09/2024	Arnoud Visser	3. In place image processing Photometric stereo	
2	37	12/09/2024	Arnoud Visser	4. Convolution and image filtering	
3	38	16/09/2024	Arnoud Visser	5. Feature: edge and corner	
3	38	19/09/2024	Arnoud Visser	6. Motion and optical flow	
4	39	23/09/2024	Martin Oswald	7. Global transform, Image stitching, alignment	
4	39	26/09/2024	Martin Oswald	8. Basics of Recognition 1	Online Lectures
5	40	30/09/2024	Martin Oswald	9. Basics of Recognition 2	
5	40	03/10/2024	Martin Oswald	10. Convolutional neural networks (Basic)	
6	41	07/10/2024	Martin Oswald	Today 11. Convolutional neural network (Training and testing)	
6	41	10/10/2024	Martin Oswald	12. Stereo, 3D reconstruction	
7	42	14/10/2024	TBA	13. Guest lectures	
7	42	17/10/2024	TBA; Martin & Arnoud	14. Guest lecture; Course Wrap-up, Exam Q&A	
8	43	21/10/2024	Your turn!	Final exam	

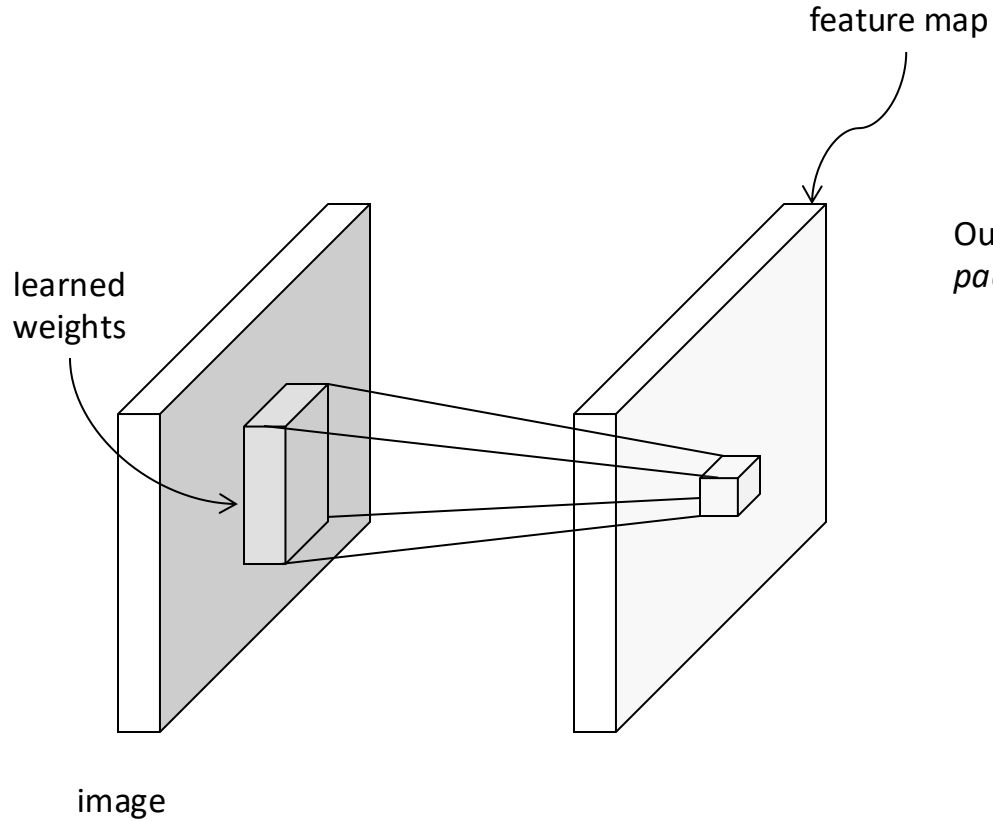
Today

Today's Agenda

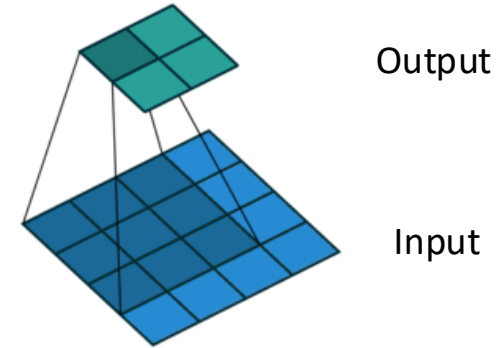
- Learning region proposals - Faster R-CNN
- Feature Pyramid Network (FPN)
- Single Shot Detectors (SSD) / Yolo
- Transfer learning
- Network architectures
 - ResNet
 - U-Net
 - Autoencoders

Recap

Convolutional Architecture

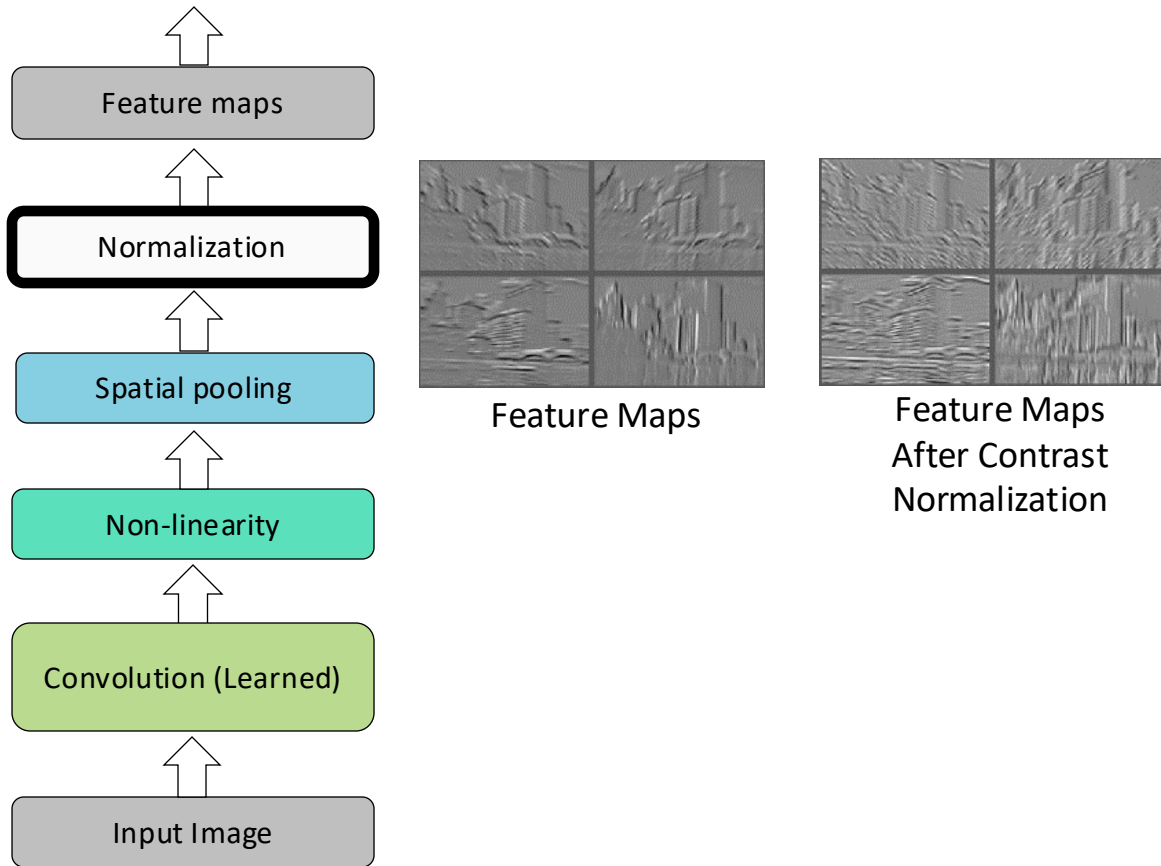


Output feature map resolution depends on *padding* and *stride*



No padding, stride 1

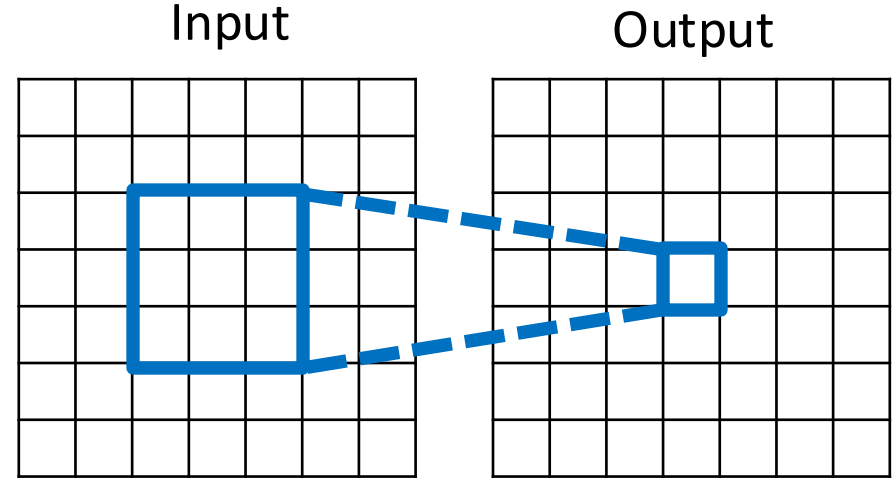
Convolutional Neural Networks



Receptive Field

3x3 convolutions, stride 1

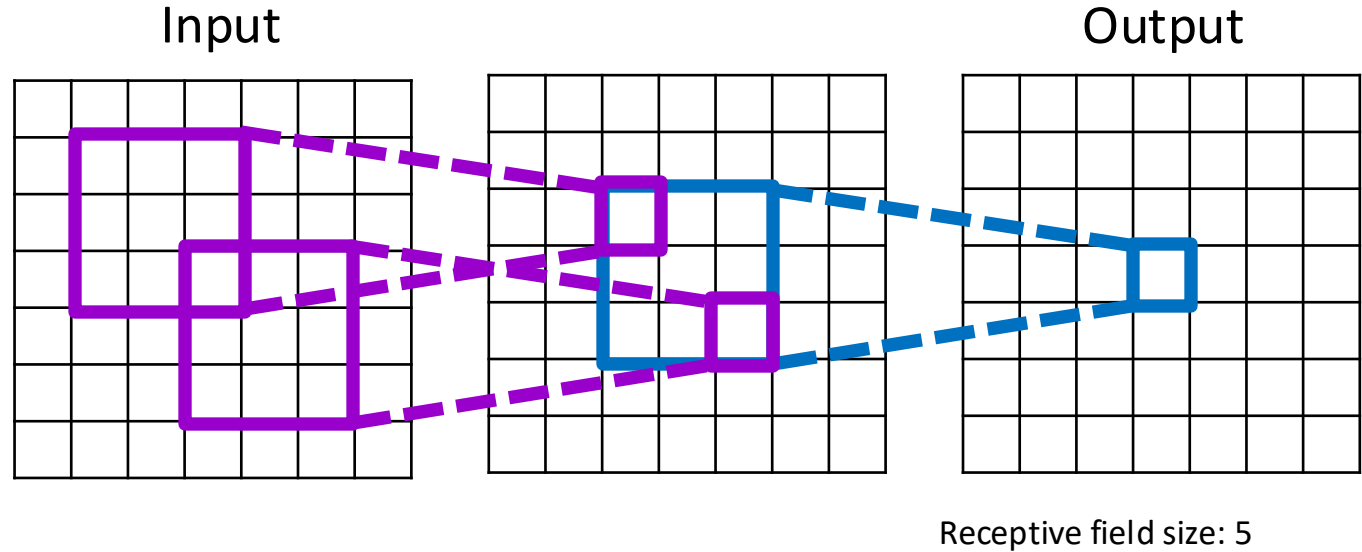
The *receptive field* of a unit is the region of the input feature map whose values contribute to the response of that unit (either in the previous layer or in the initial image)



Receptive field size: 3

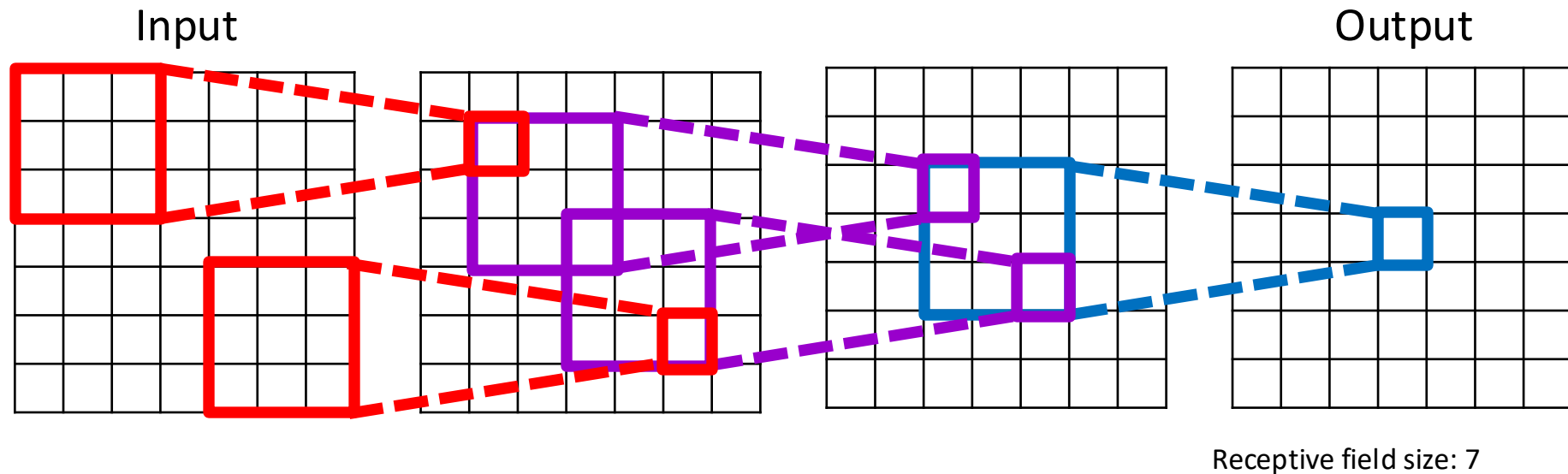
Receptive Field

3x3 convolutions, stride 1



Receptive Field

3x3 convolutions, stride 1



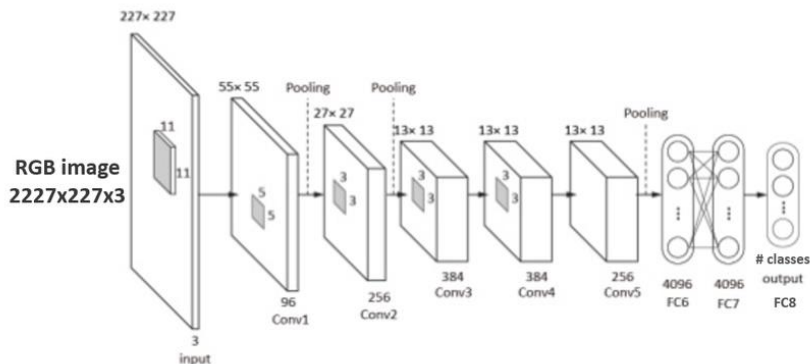
Each successive convolution adds $F - 1$ to the receptive field size

With L layers the receptive field size is $1 + L * (F - 1)$

ConvNet Architectures

AlexNet (Alex Krishevsky, 2012)

2012 ImageNet Large Scale Visual Recognition Challenge: 15% top 5 error



ConvNet Architecture:

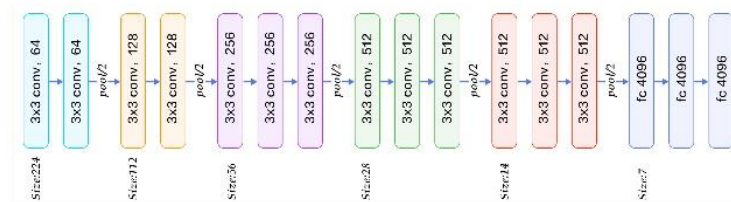
- 5 convolutional layers and 3 fully connected layers (1000 classes)
- reduces spacial dimensions: 227x227 (image) $\rightarrow \dots \rightarrow 13 \times 13$
- increases depth: 3 (image RGB depth) $\rightarrow \dots \rightarrow 256$ (nr of filters)
- use of Graphical Processing Unit (GPU)

ConvNet Architectures

VGGNet (Zimmerman, Simonyan, 2014)

The VGGNet achieved almost 92.7% top-5 test accuracy in 2014 ImageNet

VGGNet (2014)



VGGNet 16

CNN Models

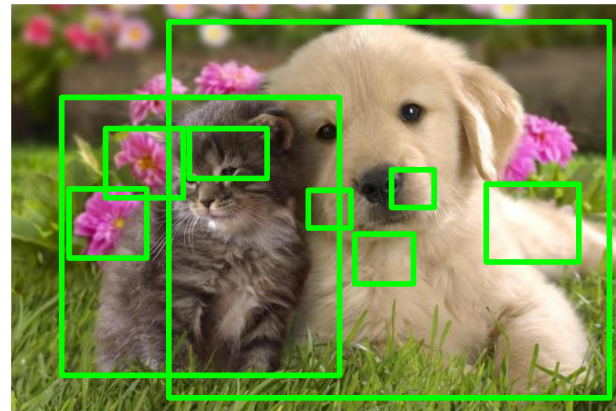
Dr. Mohamed Loey

ConvNet Architecture:

- VGG16: 13 convolutional layers and 3 fully connected layers (1000 classes)
- reduces spatial dimensions: 224x224 (image) $\rightarrow \dots \rightarrow 7 \times 7$
- increases depth: 3 (image RGB depth) $\rightarrow \dots \rightarrow 512$ (nr of filters)
- Large network. Even larger version: VGG19

Region Proposals

- Find a small set of boxes that are likely to cover all objects
- Often based on heuristics: e.g. look for “blob-like” image regions
- Relatively fast to run; e.g. Selective Search gives 2000 region proposals in a few seconds on CPU



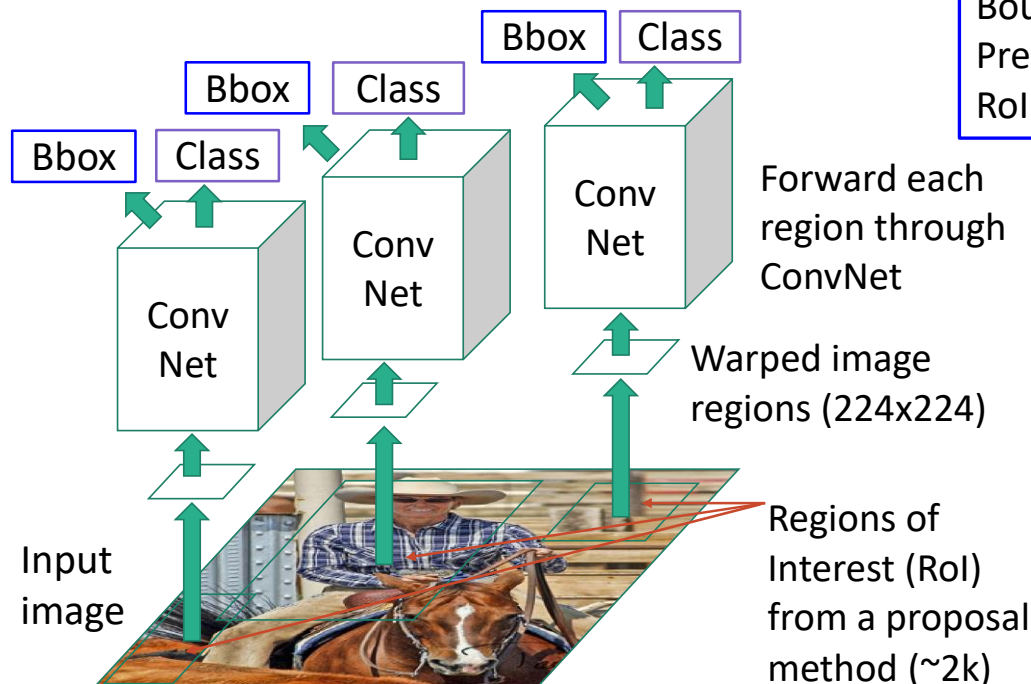
Alexe et al, “Measuring the objectness of image windows”, TPAMI 2012

Uijlings et al, “Selective Search for Object Recognition”, IJCV 2013

Cheng et al, “BING: Binarized normed gradients for objectness estimation at 300fps”, CVPR 2014

Zitnick and Dollar, “Edge boxes: Locating object proposals from edges”, ECCV 2014

Region-based CNN (R-CNN)



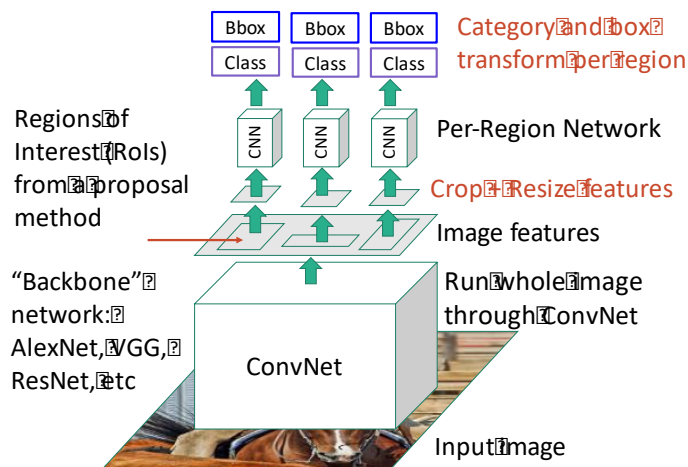
Classify each region

Bounding box regression:
Predict "transform" to correct the
RoI: 4 numbers (t_x , t_y , t_h , t_w)

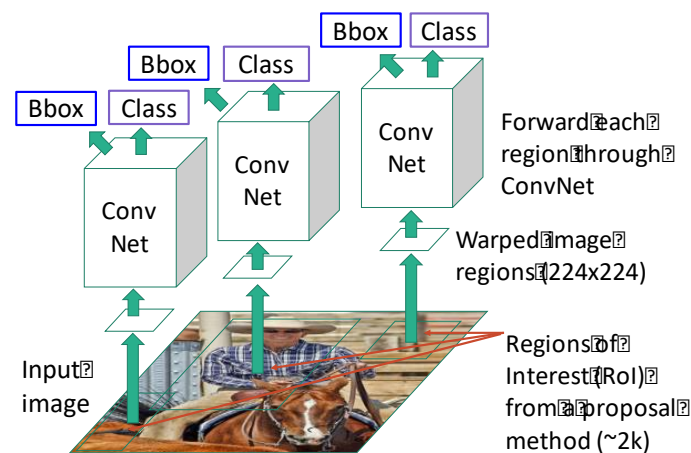
Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

Fast R-CNN vs. (Slow) R-CNN

Fast R-CNN: Apply differentiable cropping to shared image features



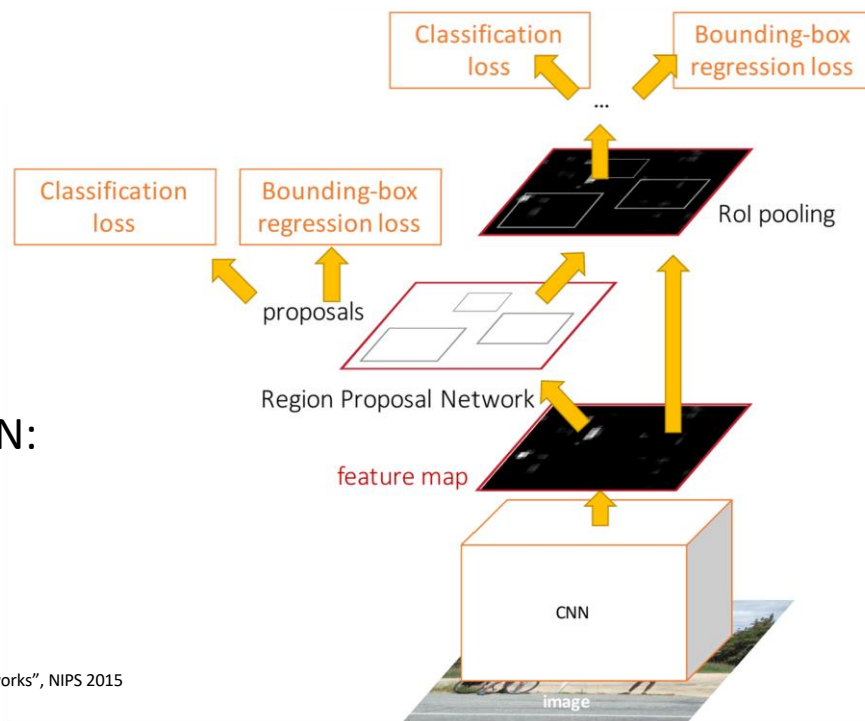
"Slow" R-CNN: Apply differentiable cropping to shared image features



Faster R-CNN: Learnable Region Proposals

Insert **Region Proposal Network (RPN)** to predict proposals from features

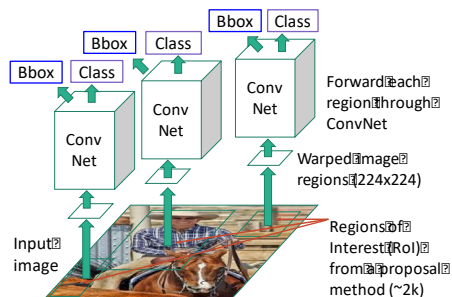
Otherwise same as Fast R-CNN:
Crop features for each proposal, classify each one



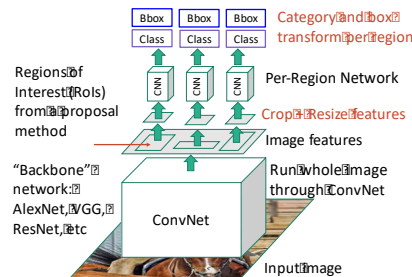
Ren et al, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks", NIPS 2015
Figure copyright 2015, Ross Girshick; reproduced with permission

Overview: R-CNN Variants

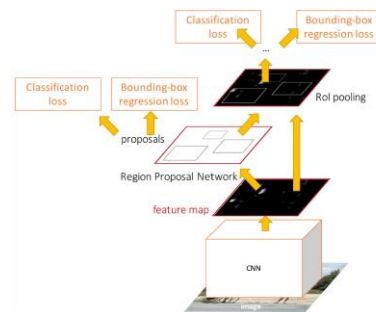
“Slow” R-CNN: Run CNN independently for each region



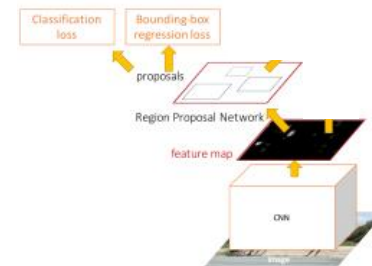
Fast R-CNN: Apply differentiable cropping to shared image features



Faster R-CNN: Compute proposals with CNN



Single-Stage: Fully convolutional detector



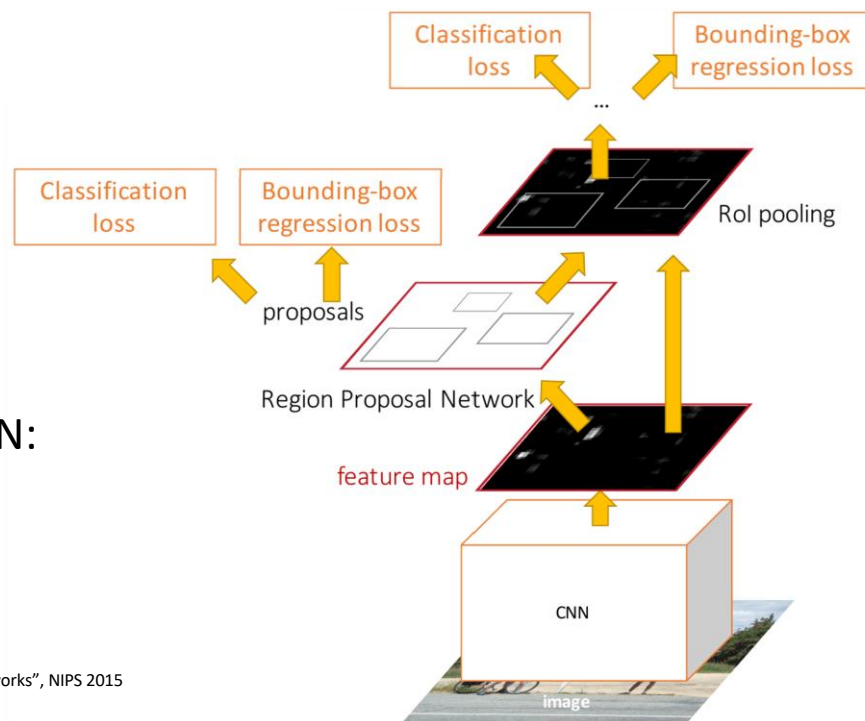
Today's Agenda

- Learning region proposals - Faster R-CNN
- Feature Pyramid Network (FPN)
- Single Shot Detectors (SSD) / Yolo
- Transfer learning
- Network architectures
 - ResNet
 - U-Net
 - Autoencoders

Faster R-CNN: Learnable Region Proposals

Insert **Region Proposal Network (RPN)** to predict proposals from features

Otherwise same as Fast R-CNN:
Crop features for each proposal, classify each one



Ren et al, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks", NIPS 2015
Figure copyright 2015, Ross Girshick; reproduced with permission

Region Proposal Network (RPN)

Run backbone CNN to get
features aligned to input image



Input Image
(e.g. 3 x 640 x 480)

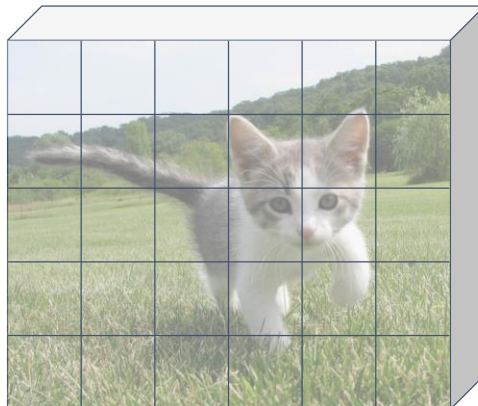


Image features
(e.g. 512 x 5 x 6)

Ren et al, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks", NIPS 2015

Region Proposal Network (RPN)

Run backbone CNN to get
features aligned to input image



Input Image
(e.g. 3 x 640 x 480)



Each feature corresponds
to a point in the input

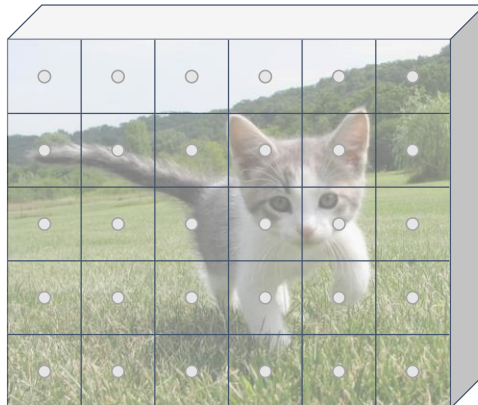
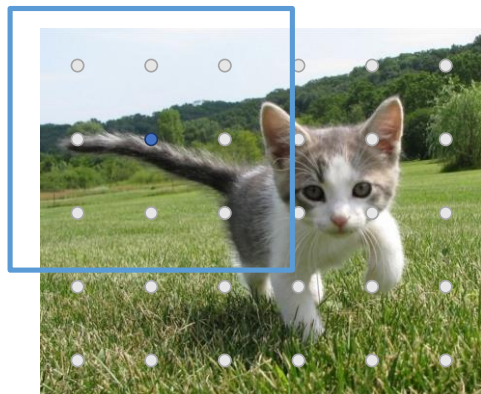


Image features
(e.g. 512 x 5 x 6)

Region Proposal Network (RPN)

Imagine an **anchor box**
of fixed size at each
point in the feature map

Run backbone CNN to get
features aligned to input image



Input Image
(e.g. 3 x 640 x 480)



Each feature corresponds
to a point in the input

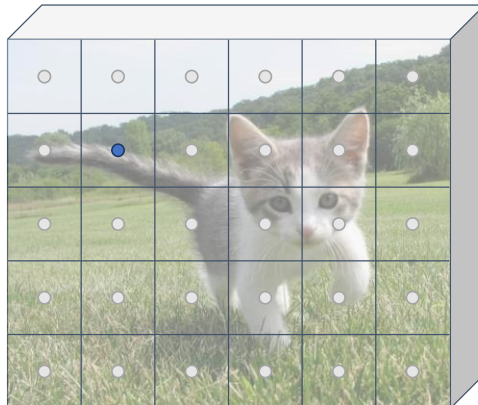
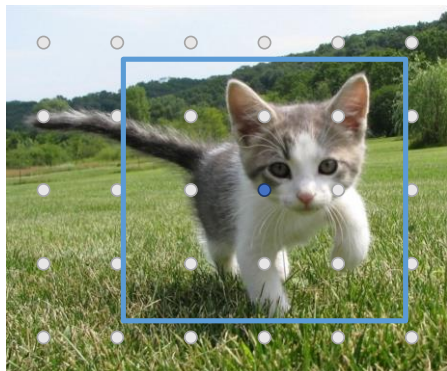


Image features
(e.g. 512 x 5 x 6)

Region Proposal Network (RPN)

Imagine an **anchor box**
of fixed size at each
point in the feature map

Run backbone CNN to get
features aligned to input image



Input Image
(e.g. 3 x 640 x 480)



Each feature corresponds
to a point in the input

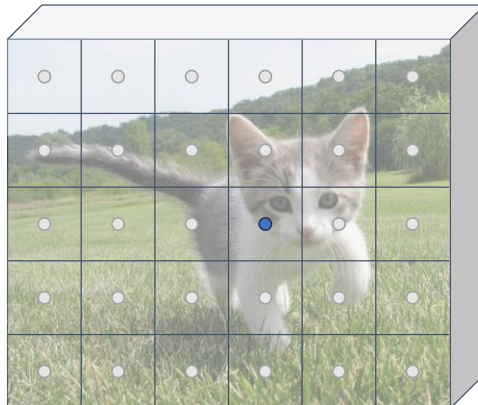
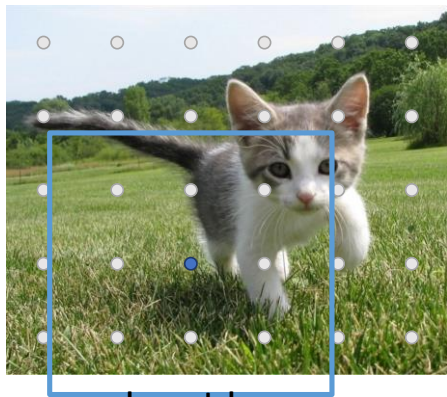


Image features
(e.g. 512 x 5 x 6)

Region Proposal Network (RPN)

Run backbone CNN to get
features aligned to input image



Input Image
(e.g. 3 x 640 x 480)



Each feature corresponds
to a point in the input

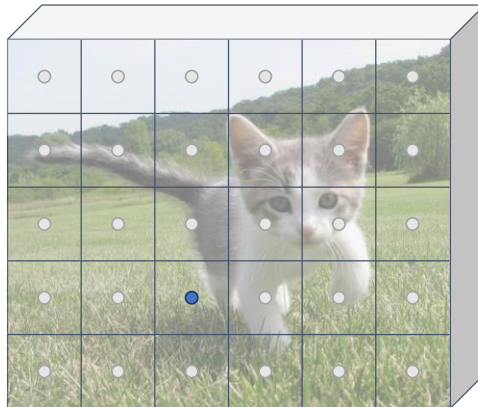
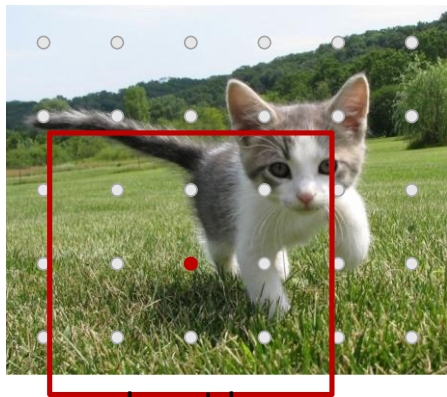


Image features
(e.g. 512 x 5 x 6)

Imagine an **anchor box**
of fixed size at each
point in the feature map

Region Proposal Network (RPN)

Run backbone CNN to get
features aligned to input image



Input Image
(e.g. 3 x 640 x 480)



Each feature corresponds
to a point in the input

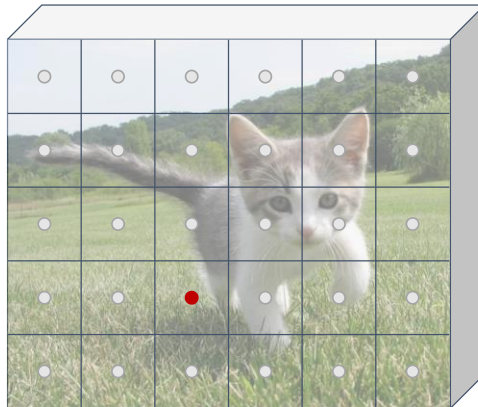


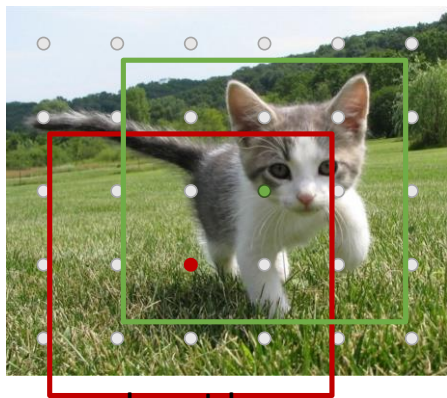
Image features
(e.g. 512 x 5 x 6)

Imagine an **anchor box**
of fixed size at each
point in the feature map

Classify each anchor as
positive (object) or
negative (no object)

Region Proposal Network (RPN)

Run backbone CNN to get
features aligned to input image



Input Image
(e.g. 3 x 640 x 480)



Each feature corresponds
to a point in the input

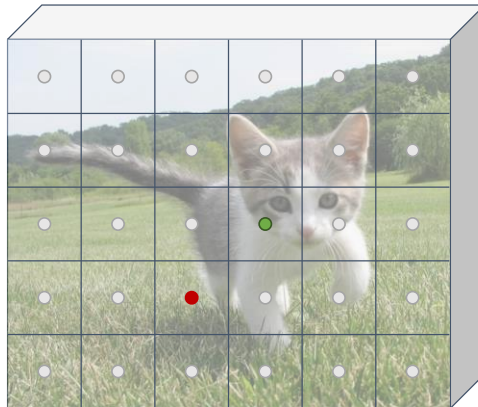


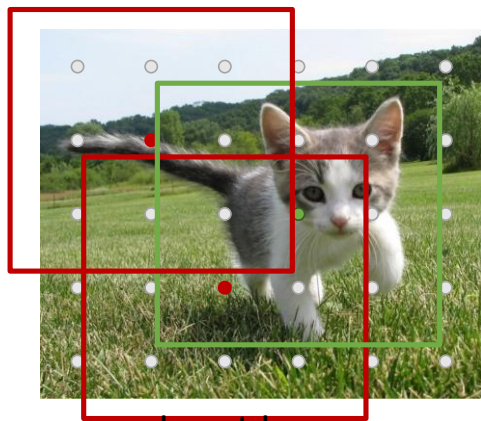
Image features
(e.g. 512 x 5 x 6)

Imagine an **anchor box**
of fixed size at each
point in the feature map

Classify each anchor as
positive (object) or
negative (no object)

Region Proposal Network (RPN)

Run backbone CNN to get
features aligned to input image



Input Image
(e.g. 3 x 640 x 480)



Each feature corresponds
to a point in the input

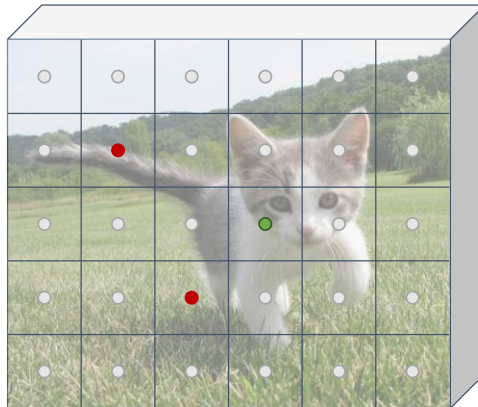
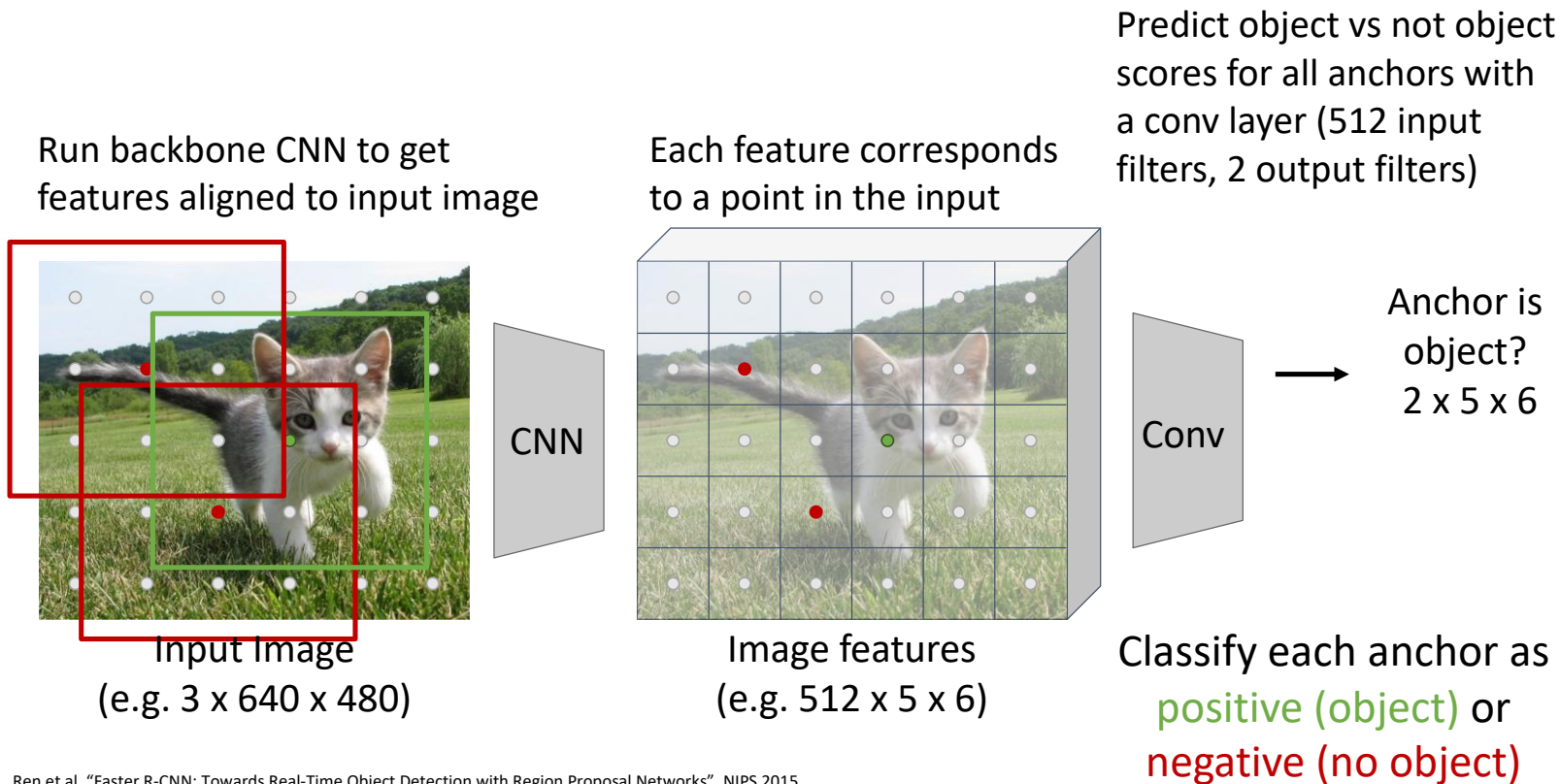


Image features
(e.g. 512 x 5 x 6)

Imagine an **anchor box**
of fixed size at each
point in the feature map

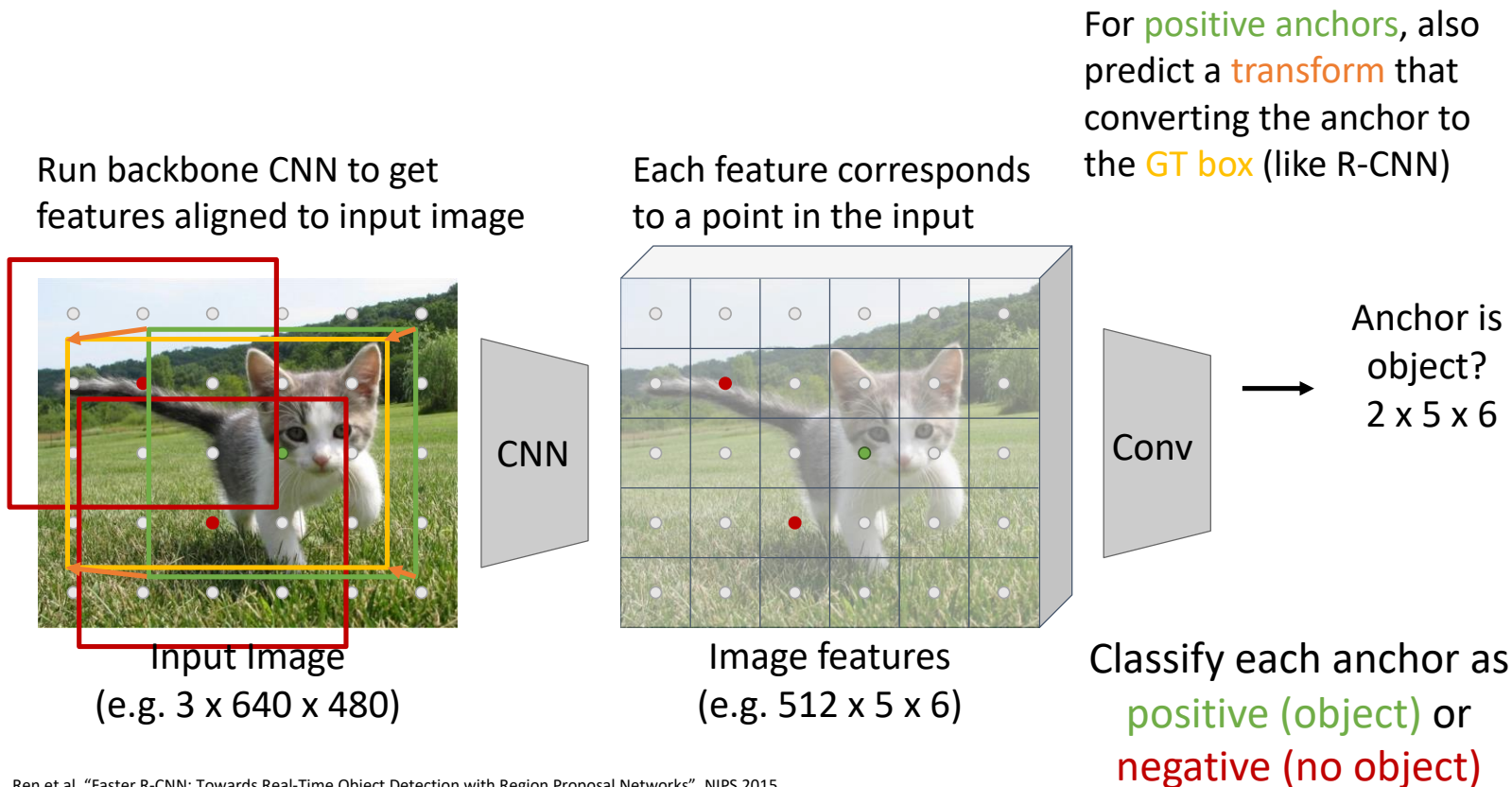
Classify each anchor as
positive (object) or
negative (no object)

Region Proposal Network (RPN)



Ren et al, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks", NIPS 2015

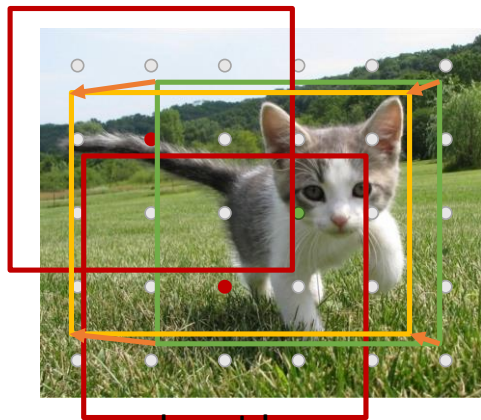
Region Proposal Network (RPN)



Ren et al, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks", NIPS 2015

Region Proposal Network (RPN)

Run backbone CNN to get
features aligned to input image



Input Image
(e.g. 3 x 640 x 480)



Each feature corresponds
to a point in the input

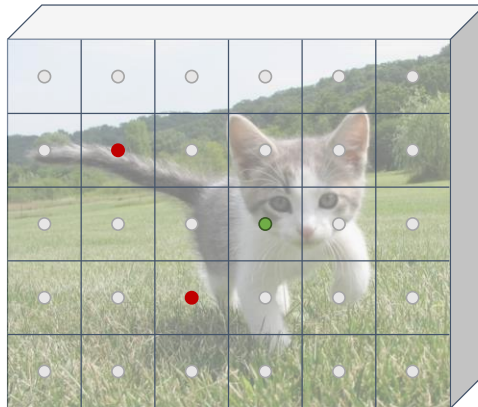


Image features
(e.g. 512 x 5 x 6)

For **positive anchors**, also
predict a **transform** that
converting the anchor to
the **GT box** (like R-CNN)
Predict transforms with conv



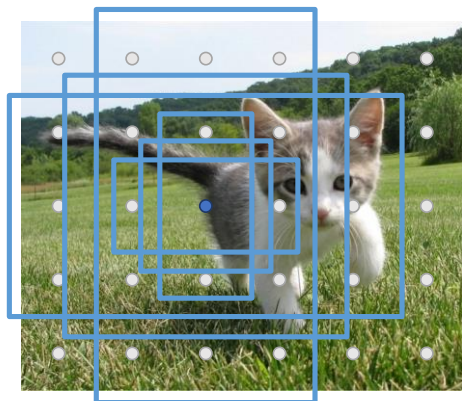
Anchor is
object?
2 x 5 x 6

Anchor
transforms
4 x 5 x 6

Classify each anchor as
positive (object) or
negative (no object)

Region Proposal Network (RPN)

Run backbone CNN to get features aligned to input image



Input Image
(e.g. 3 x 640 x 480)



Each feature corresponds to a point in the input

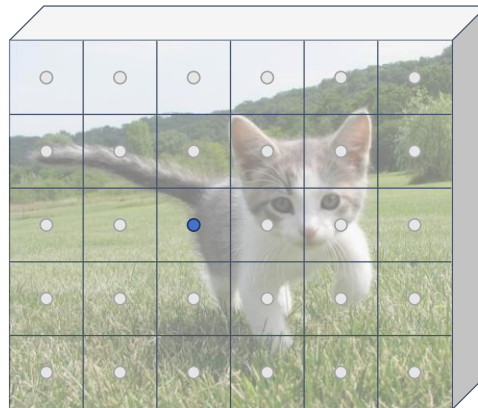


Image features
(e.g. 512 x 5 x 6)



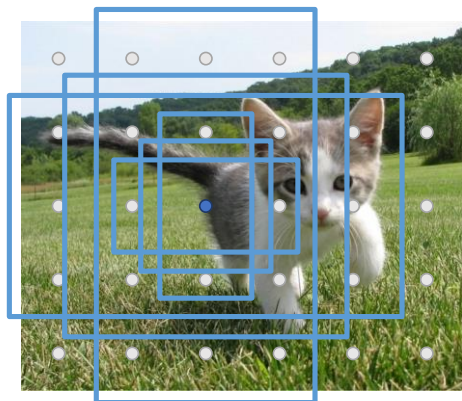
In practice: Rather than using one anchor per point, instead consider K different anchors with different size and scale (here $K = 6$)

Anchor is
object?
 $2K \times 5 \times 6$

Anchor
transforms
 $4K \times 5 \times 6$

Region Proposal Network (RPN)

Run backbone CNN to get features aligned to input image



Input Image
(e.g. 3 x 640 x 480)



Each feature corresponds to a point in the input

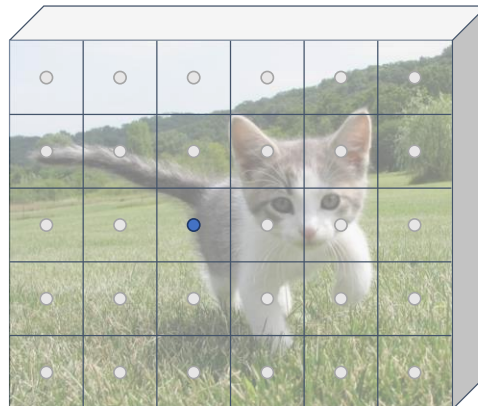


Image features
(e.g. 512 x 5 x 6)

In practice: Rather than using one anchor per point, instead consider K different anchors with different size and scale (here $K = 6$)



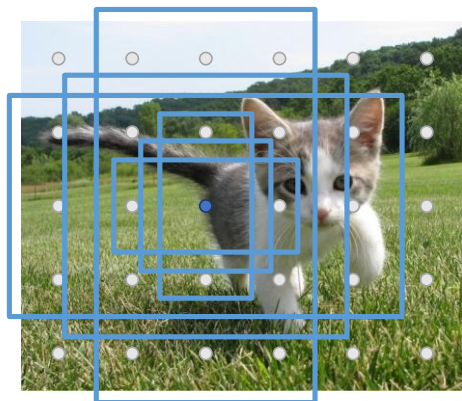
Anchor is object?
 $2K \times 5 \times 6$

Anchor transforms
 $4K \times 5 \times 6$

During training, supervised positive / negative anchors and box transforms like R-CNN

Region Proposal Network (RPN)

Run backbone CNN to get features aligned to input image



Input Image
(e.g. 3 x 640 x 480)



Each feature corresponds to a point in the input

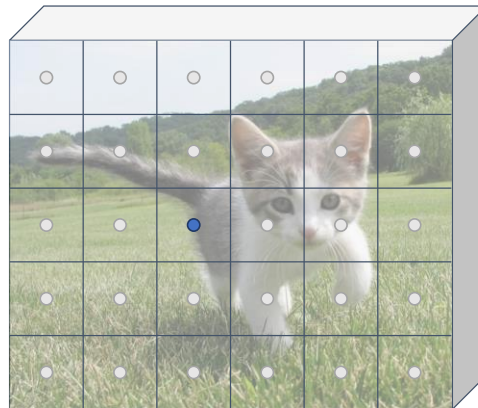


Image features
(e.g. 512 x 5 x 6)

In practice: Rather than using one anchor per point, instead consider K different anchors with different size and scale (here K = 6)



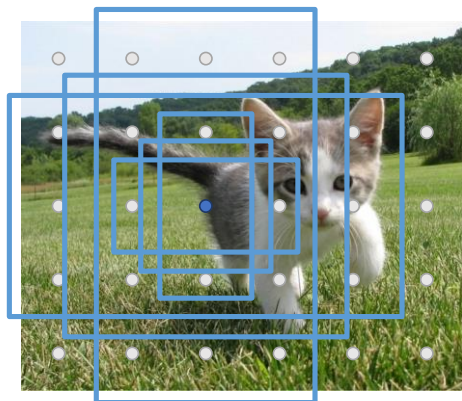
Anchor is
object?
2K x 5 x 6

Anchor
transforms
4K x 5 x 6

Positive anchors: ≥ 0.7 IoU with some GT box (plus highest IoU to each GT)

Region Proposal Network (RPN)

Run backbone CNN to get features aligned to input image



Input Image
(e.g. 3 x 640 x 480)



Each feature corresponds to a point in the input

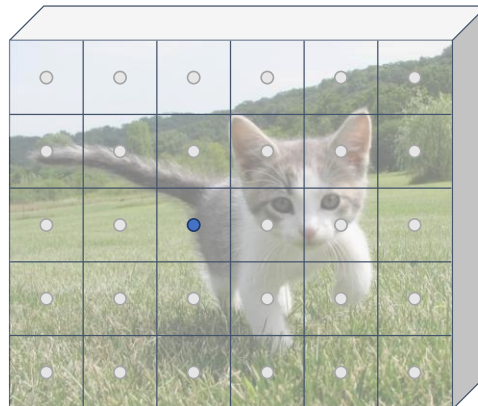


Image features
(e.g. 512 x 5 x 6)

In practice: Rather than using one anchor per point, instead consider K different anchors with different size and scale (here K = 6)



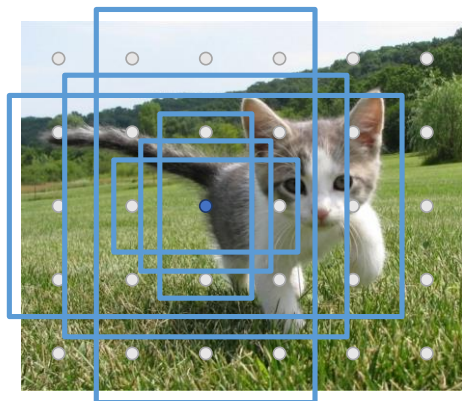
Anchor is
object?
2K x 5 x 6

Anchor
transforms
4K x 5 x 6

Negative anchors: < 0.3 IoU with all GT boxes. Don't supervised transforms for negative boxes.

Region Proposal Network (RPN)

Run backbone CNN to get features aligned to input image



Input Image
(e.g. 3 x 640 x 480)



Each feature corresponds to a point in the input

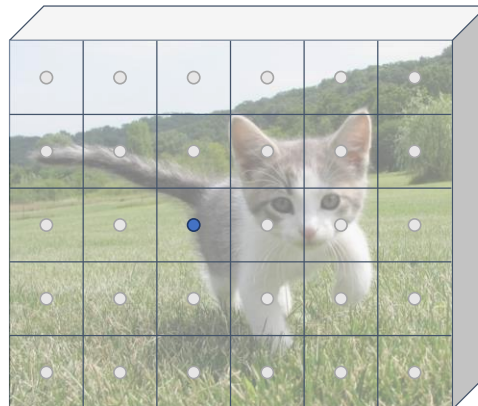


Image features
(e.g. 512 x 5 x 6)

In practice: Rather than using one anchor per point, instead consider K different anchors with different size and scale (here K = 6)



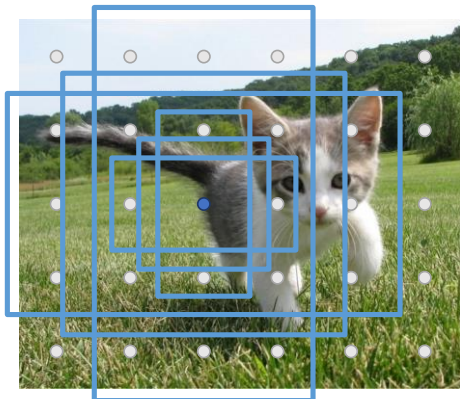
Anchor is
object?
2K x 5 x 6

Anchor
transforms
4K x 5 x 6

Neutral anchors: between 0.3 and 0.7 IoU with all GT boxes; ignored during training

Region Proposal Network (RPN)

Run backbone CNN to get features aligned to input image



Input Image
(e.g. 3 x 640 x 480)



Each feature corresponds to a point in the input

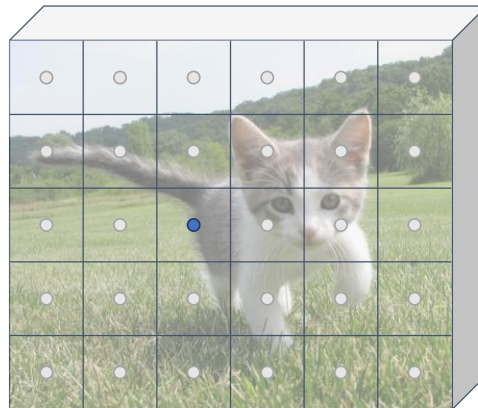


Image features
(e.g. 512 x 5 x 6)

In practice: Rather than using one anchor per point, instead consider K different anchors with different size and scale (here $K = 6$)



Anchor is
object?
 $2K \times 5 \times 6$

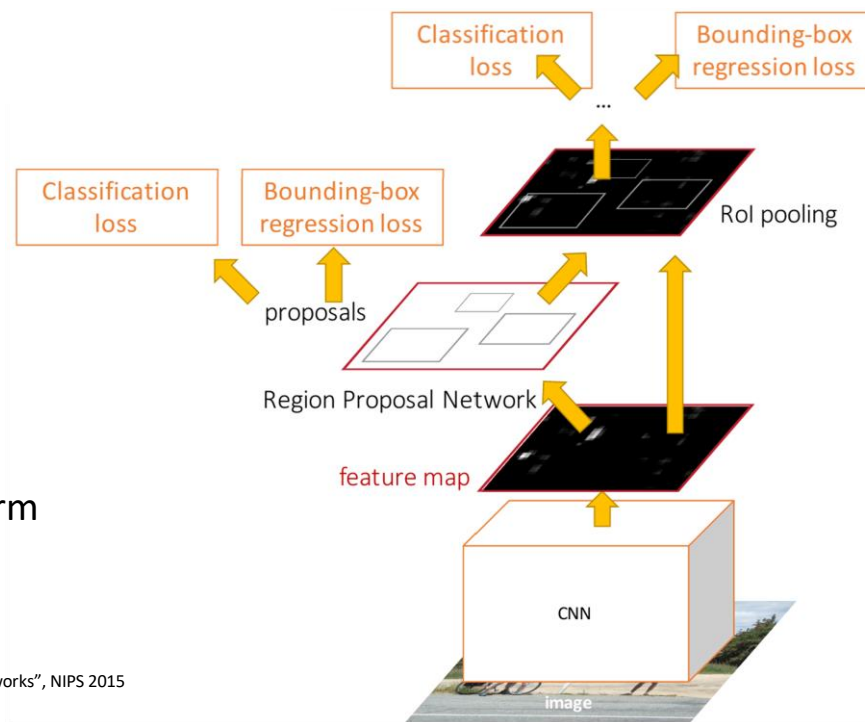
Anchor
transforms
 $4K \times 5 \times 6$

At test-time, sort all $K \times 5 \times 6$ boxes by their positive score, take top 300 as our region proposals

Faster R-CNN: Learnable Region Proposals

Jointly train with 4 losses:

1. **RPN classification:** anchor box is object / not an object
2. **RPN regression:** predict transform from anchor box to proposal box
3. **Object classification:** classify proposals as background / object class
4. **Object regression:** predict transform from proposal box to object box



Ren et al, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks", NIPS 2015
Figure copyright 2015, Ross Girshick; reproduced with permission

Faster R-CNN: Learnable Region Proposals

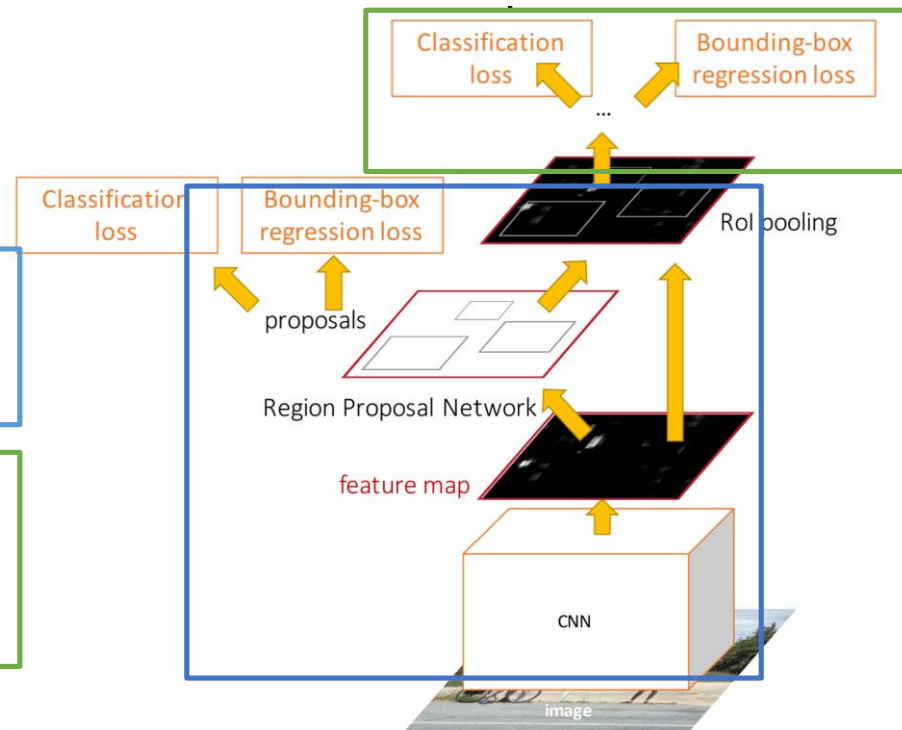
Faster R-CNN is a
Two-stage object detector

First stage: Run once per image

- Backbone network
- Region proposal network

Second stage: Run once per region

- Crop features: RoI pool / align
- Predict object class
- Prediction bbox offset

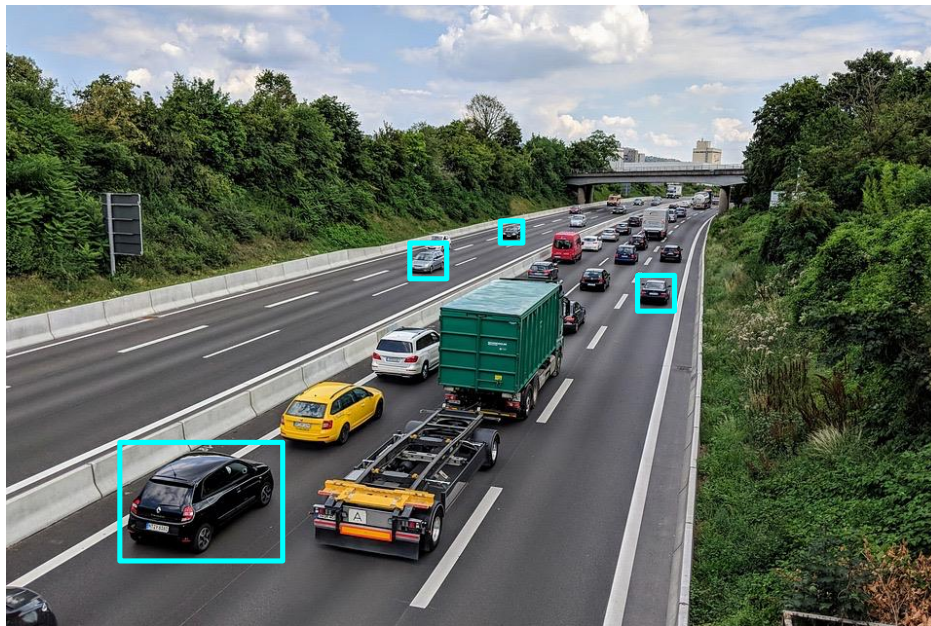


Today's Agenda

- Learning region proposals - Faster R-CNN
- Feature Pyramid Network (FPN)
- Single Shot Detectors (SSD) / Yolo
- Transfer learning
- Network architectures
 - ResNet
 - U-Net
 - Autoencoders

Dealing with Scale Variations

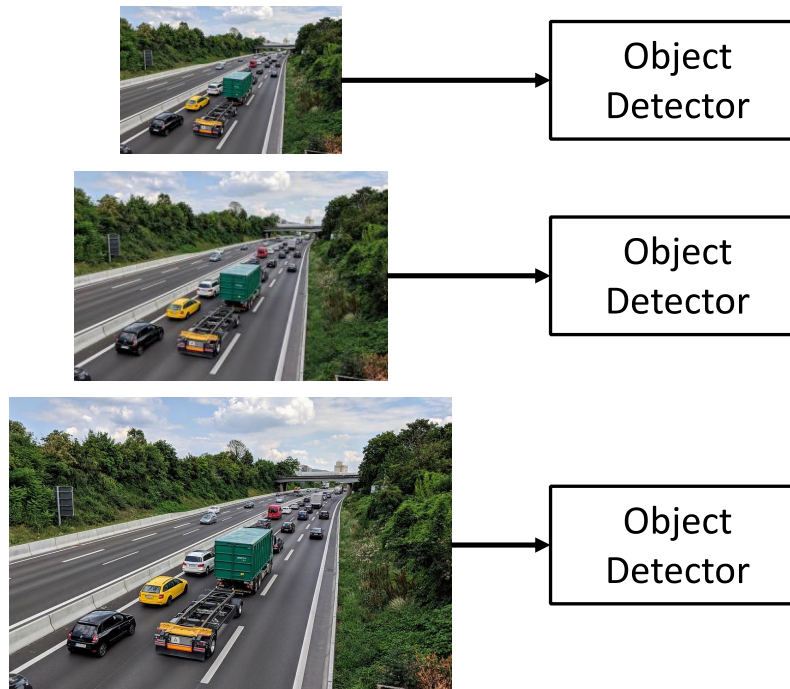
We need to detect objects of many different scales.
How to improve *scale invariance* of the detector?



[This image](#) is free for commercial use under the [Pixabay license](#)

Scale Variations: Image Pyramid

Classic idea: build an *image pyramid* by resizing the image to different scales, then process each image scale independently.



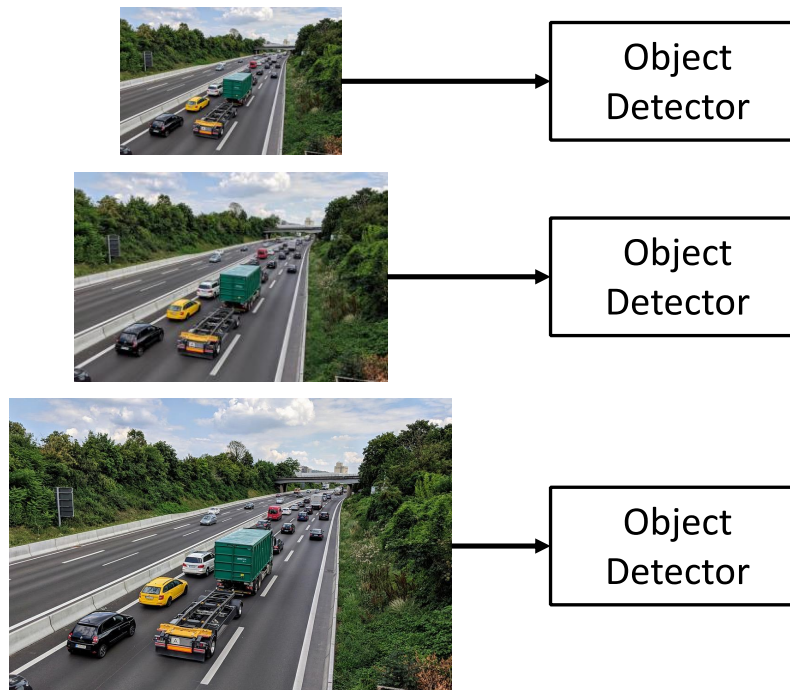
Lin et al, "Feature Pyramid Networks for Object Detection", ICCV 2017

Scale Variations: Image Pyramid

Classic idea: build an *image pyramid* by resizing the image to different scales, then process each image scale independently.

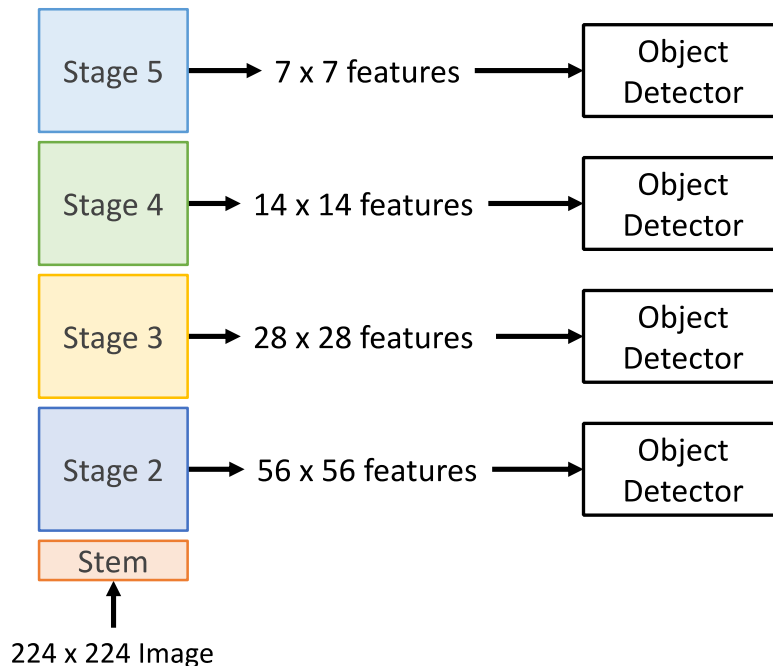
Problem: Expensive! Don't share any computation between scales

Lin et al, "Feature Pyramid Networks for Object Detection", ICCV 2017



Scale Variations: Multiscale Features

CNNs have multiple *stages* that operate at different resolutions. Attach an independent detector to the features at each level



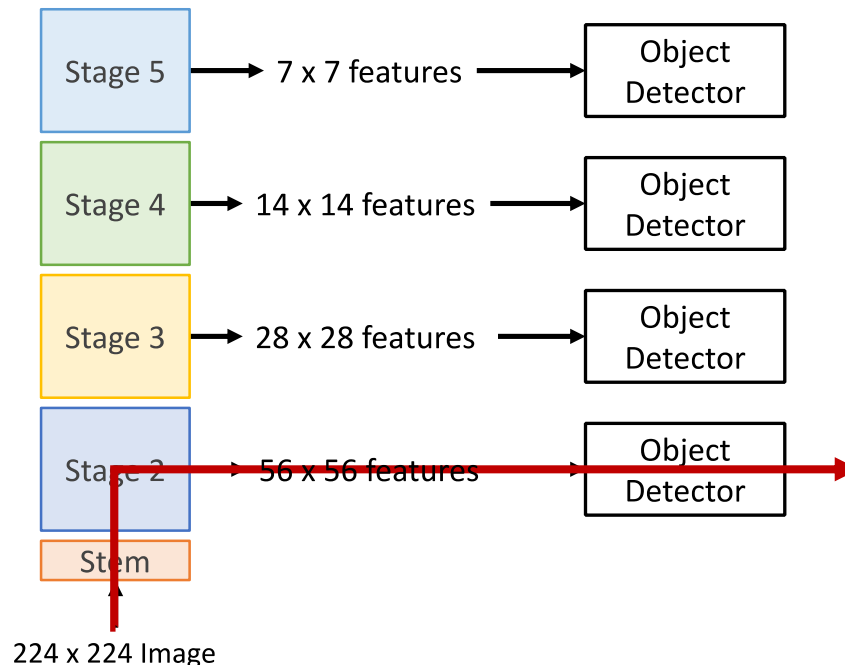
Lin et al, "Feature Pyramid Networks for Object Detection", ICCV 2017

Scale Variations: Multiscale Features

CNNs have multiple *stages* that operate at different resolutions. Attach an independent detector to the features at each level

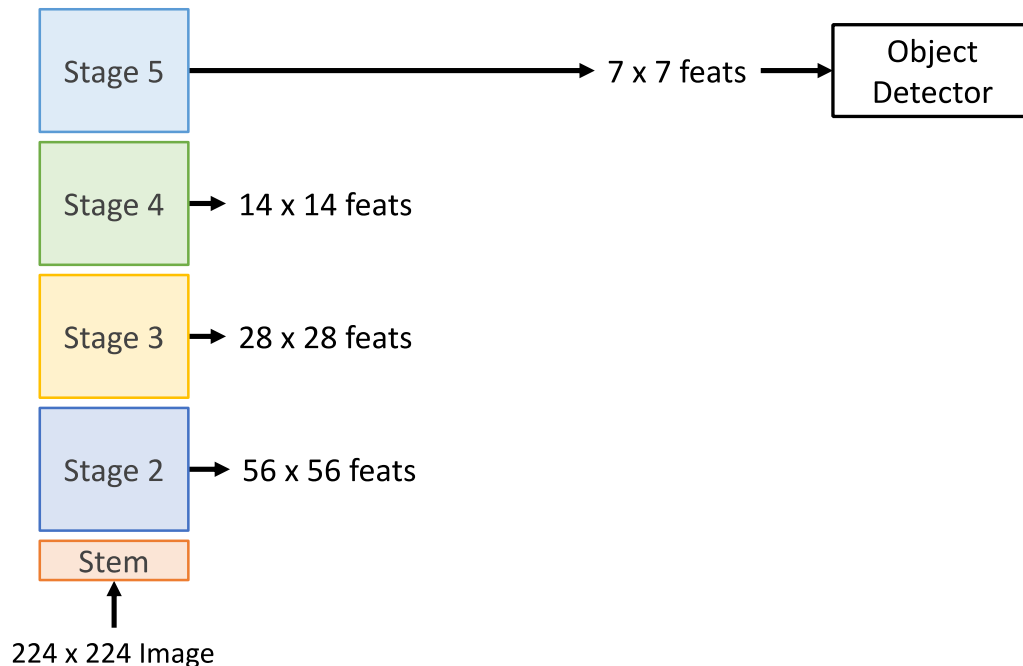
Problem: detector on early features doesn't make use of the entire backbone; doesn't get access to high-level features

Lin et al, "Feature Pyramid Networks for Object Detection", ICCV 2017



Scale Variations: Feature Pyramid Network

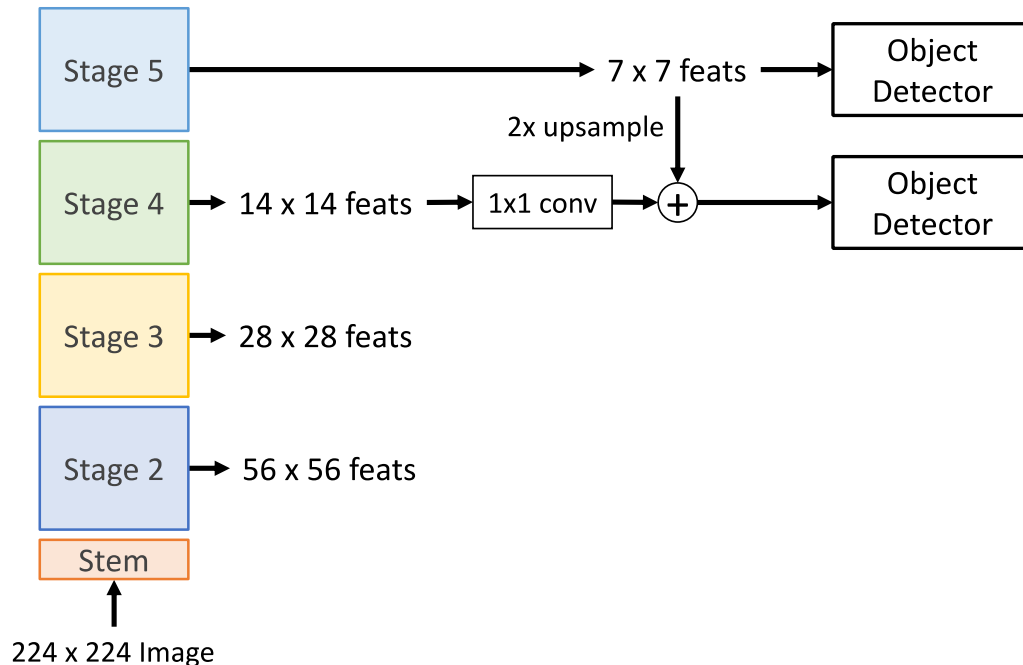
Add *top down* connections that feed information from high level features back down to lower level features



Lin et al, "Feature Pyramid Networks for Object Detection", ICCV 2017

Scale Variations: Feature Pyramid Network

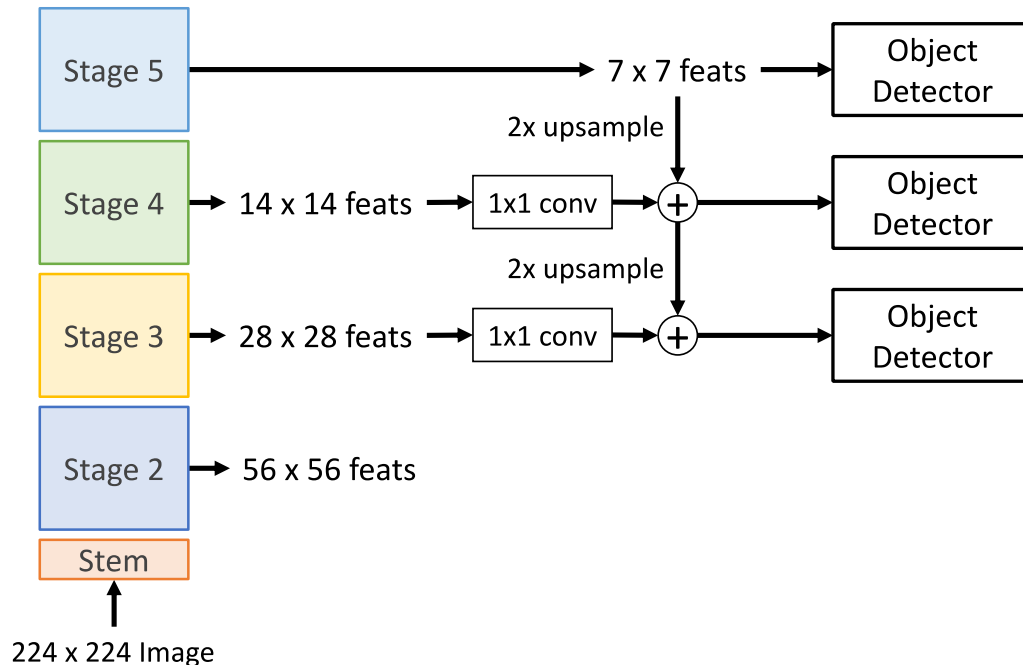
Add *top down* connections that feed information from high level features back down to lower level features



Lin et al, "Feature Pyramid Networks for Object Detection", ICCV 2017

Scale Variations: Feature Pyramid Network

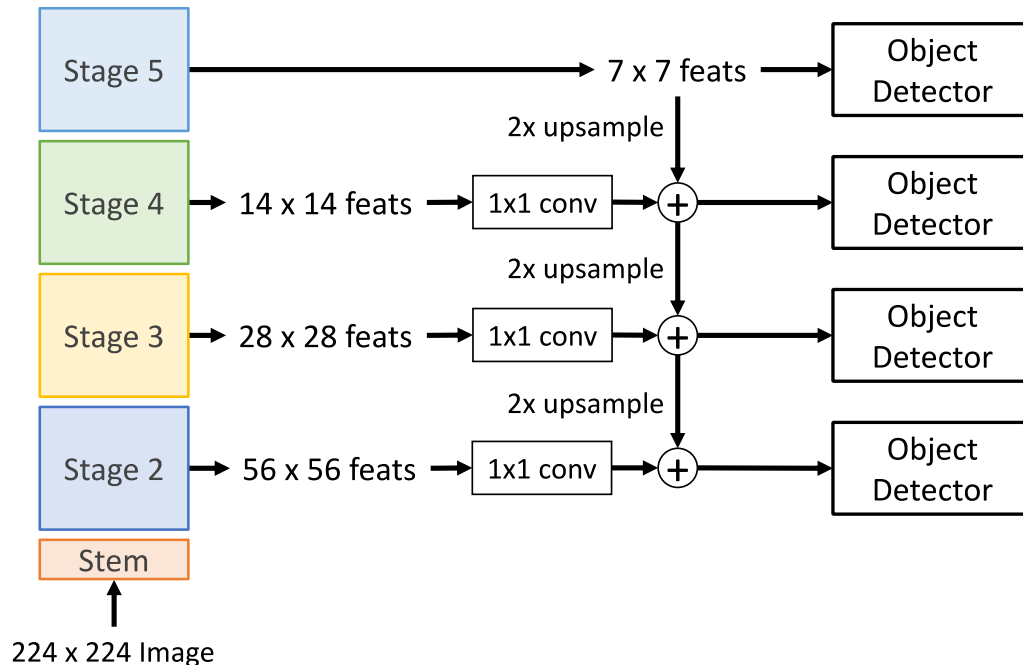
Add *top down* connections that feed information from high level features back down to lower level features



Lin et al, "Feature Pyramid Networks for Object Detection", ICCV 2017

Scale Variations: Feature Pyramid Network

Add *top down* connections that feed information from high level features back down to lower level features

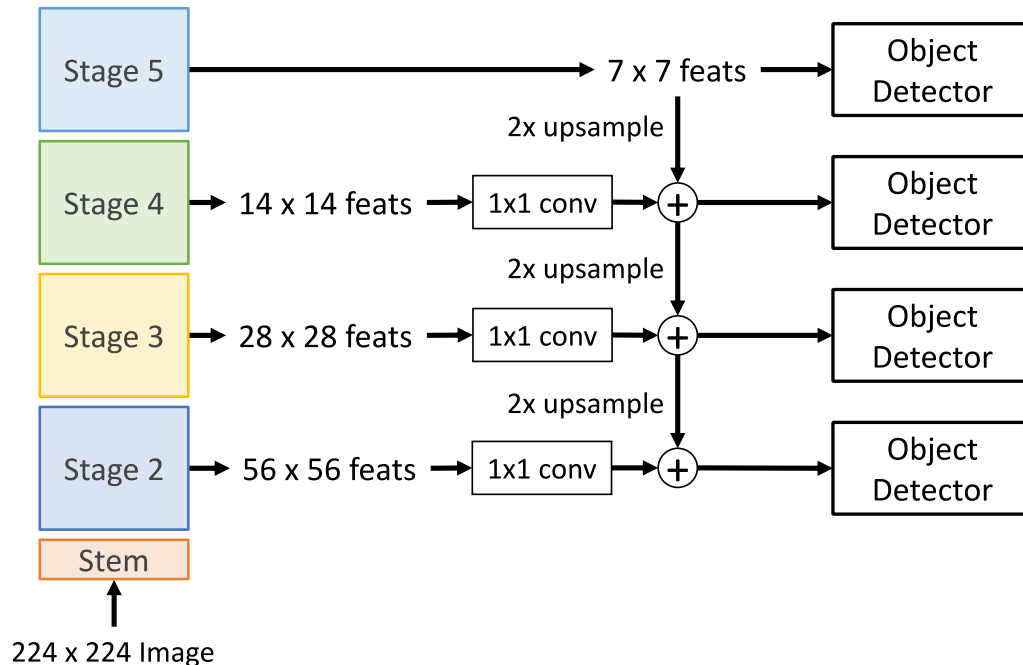


Lin et al, "Feature Pyramid Networks for Object Detection", ICCV 2017

Scale Variations: Feature Pyramid Network

Add *top down* connections that feed information from high level features back down to lower level features

Efficient multiscale features where all levels benefit from the whole backbone! Widely used in practice



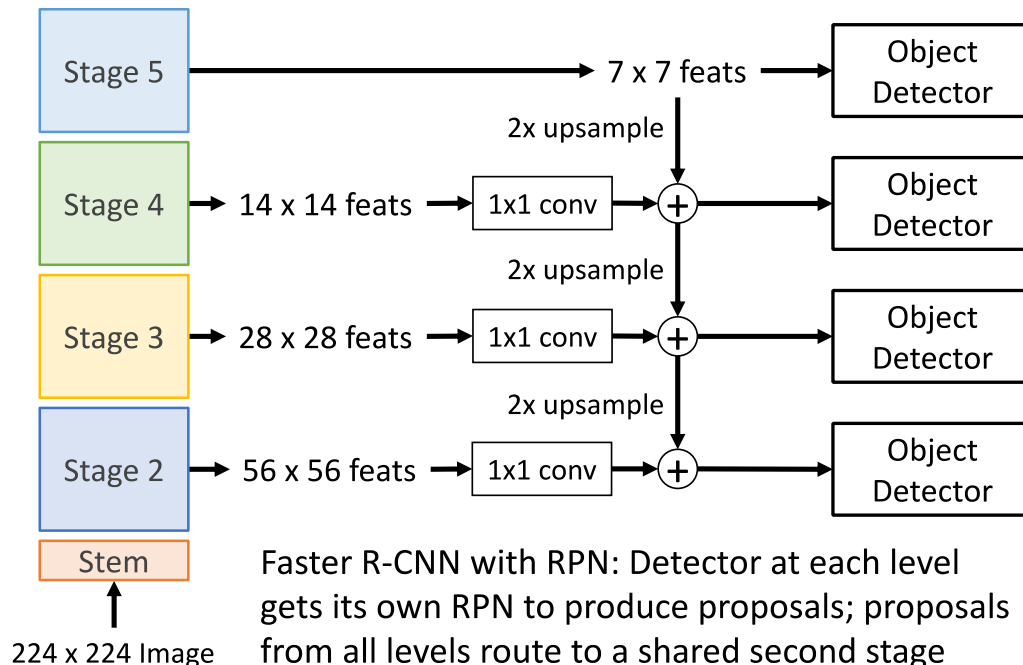
Lin et al, "Feature Pyramid Networks for Object Detection", ICCV 2017

Scale Variations: Feature Pyramid Network

Add *top down* connections that feed information from high level features back down to lower level features

Efficient multiscale features where all levels benefit from the whole backbone! Widely used in practice

Lin et al, "Feature Pyramid Networks for Object Detection", ICCV 2017



Faster R-CNN: Learnable Region Proposals

Faster R-CNN is a
Two-stage object detector

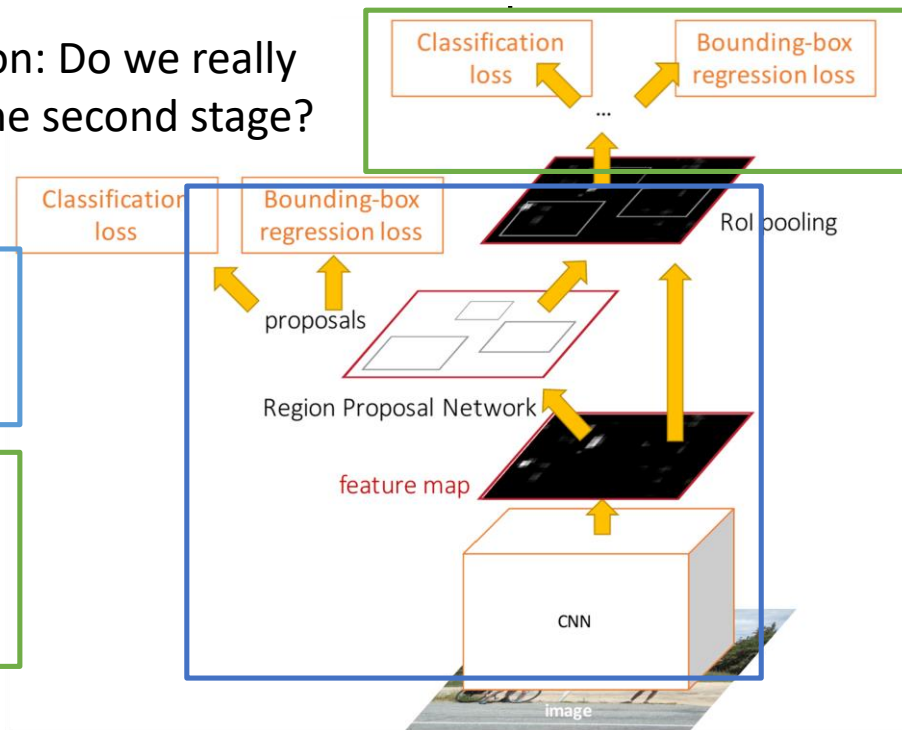
First stage: Run once per image

- Backbone network
- Region proposal network

Second stage: Run once per region

- Crop features: RoI pool / align
- Predict object class
- Prediction bbox offset

Question: Do we really
need the second stage?

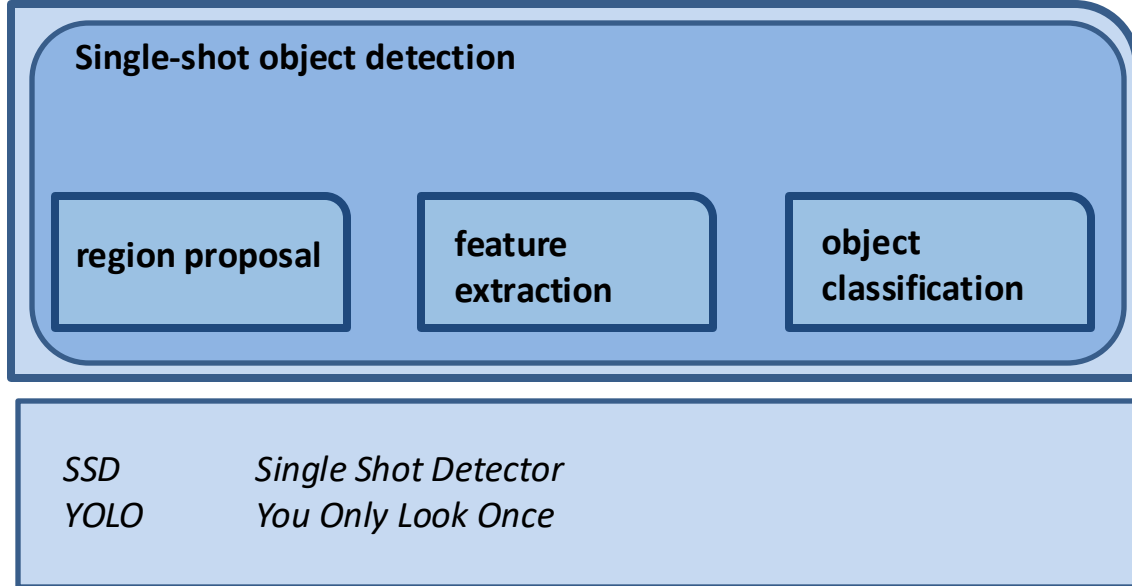


Today's Agenda

- Learning region proposals - Faster R-CNN
- Feature Pyramid Network (FPN)
- Single Shot Detectors (SSD) / Yolo
- Transfer learning
- Network architectures
 - ResNet
 - U-Net
 - Autoencoders

Single-Shot Detection

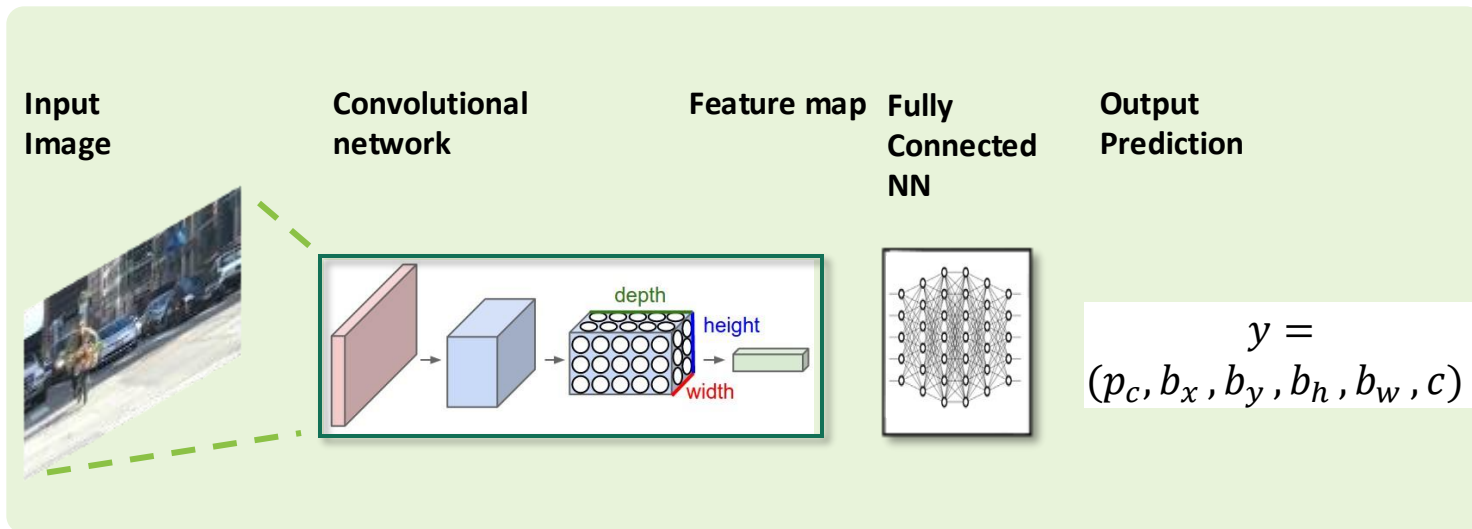
Single-Shot Detection methods do the region proposal, feature extraction and classification steps in one go



Single-Shot Detection

Single-Shot Detectors:

- Feature map is created for the whole image
- Prediction is a combination of bounding box coordinates and class confidence.



Single-Shot Detection

Anchor-based single-shot detectors work with following steps:

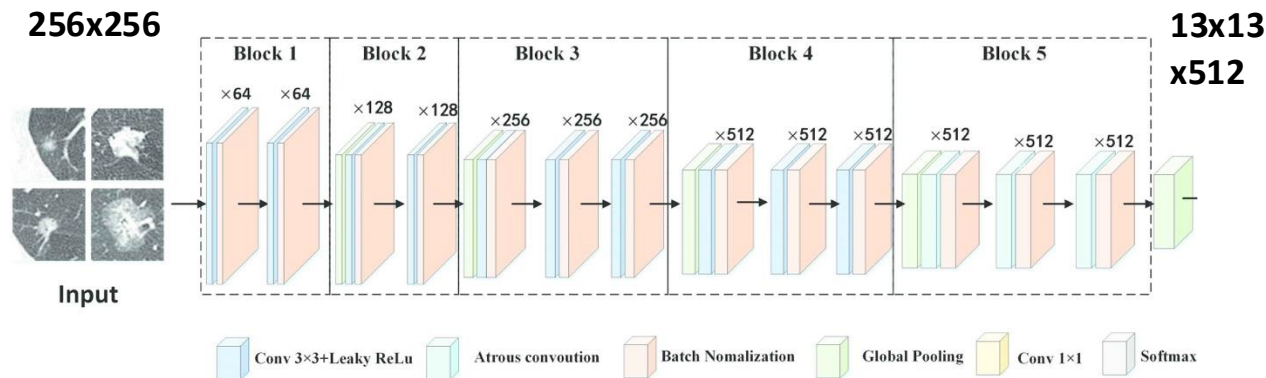
- A. Divide image in grid cells
- B. Pre-define anchor-boxes
- C. Refine box coordinates
- D. Train class confidence
- E. Filter best predictions



Single-Shot Detection

A. Divide image in grid cells

Use a convolutional neural network to divide image in grid cells and extract convolutional features for each grid cell

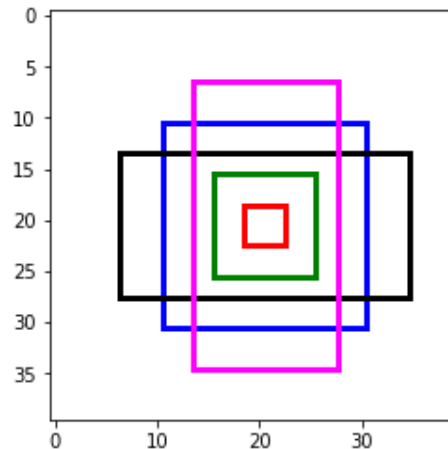
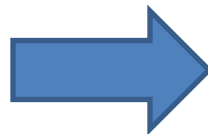
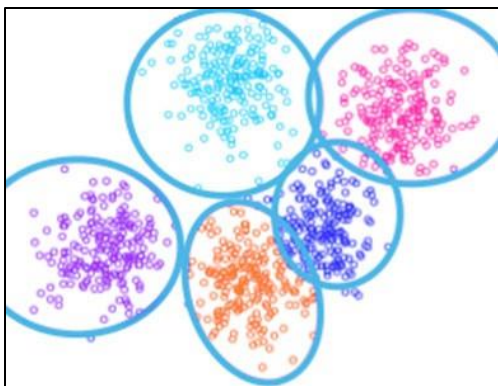


CNN typically reduces dimension in each convolutional block (by pooling)
This example CNN leads to an Feature Map with 512 features (filter outcomes) for each of 13x13 grid cells.

Single-Shot Detection

B. Pre-define anchor boxes

Define default anchor boxes with typical object sizes and aspect ratio's



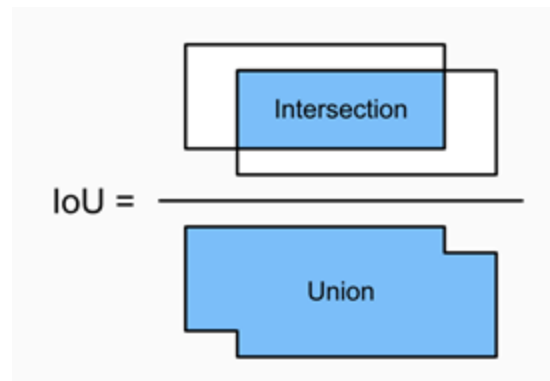
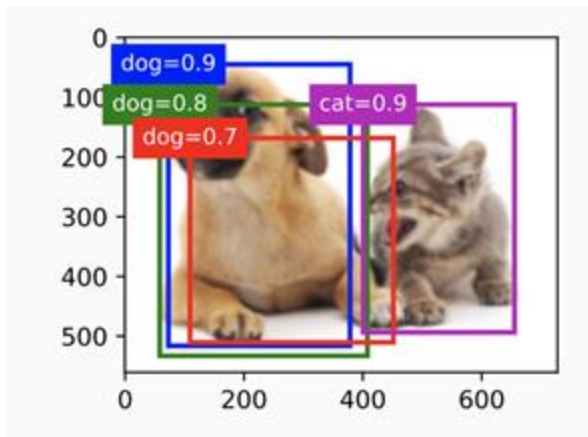
5 default anchor boxes are defined manually or based on k-means clustering on an existing image dataset. This step is usually done off-line (not included in CNN training)

Single-Shot Detection

C. Refine box coordinates

For each grid cell, a number of box sizes can be learned by adjusting the pre-defined anchor boxes.

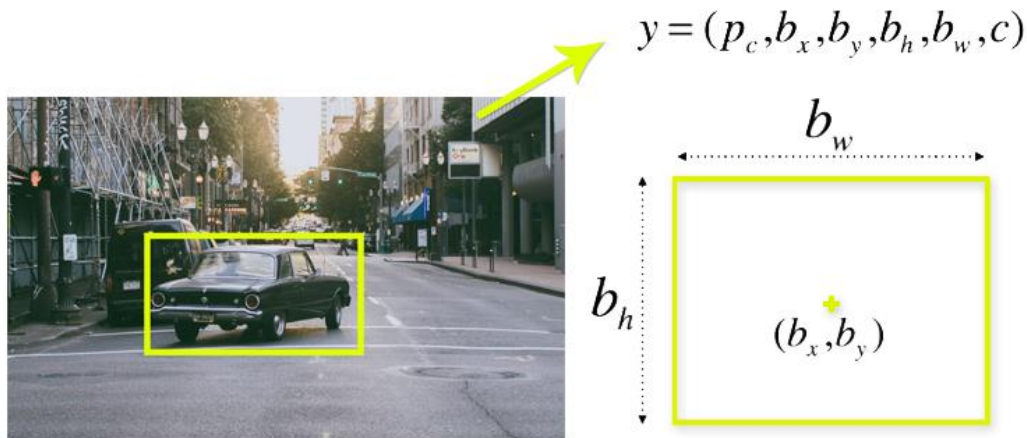
Learning is by minimizing Loss (Intersection over Union) between prediction and ground truth.



Single-Shot Detection

D. Train class confidence (jointly with step C)

Based on convolutional features for each box with an object, a class confidence can be learned.



Yolo example: class confidence is combination of c and p_c

b_x, b_y, b_h, b_w

box coordinates

c

box probability that there is an object in box

p_c

conditional class probability for each class c

Single-Shot Detection

D. Train class confidence (jointly with step C)

Location and class probability are trained simultaneously. Loss function (regression) combined both.

The diagram illustrates the loss function for training class confidence, with annotations explaining the components:

- Prediction** and **Ground truth** point to the first term of the loss function.
- Box location** points to the second term of the loss function.
- Box Confidence score (IoU)** points to the third term of the loss function.
- regulators** points to the coefficients λ_{coord} and λ_{noobj} .
- conditional class probability for class c** points to the fourth term of the loss function.

$$\begin{aligned}
 & \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\
 & + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[(\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right] \\
 & + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 \\
 & + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \\
 & + \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2
 \end{aligned}$$

Single-Shot Detection

E. Filter best predictions

Non-maximum suppression filters out predictions with low class confidence. Filter out boxes with confidence below threshold.

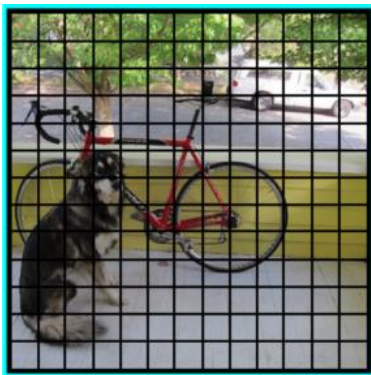
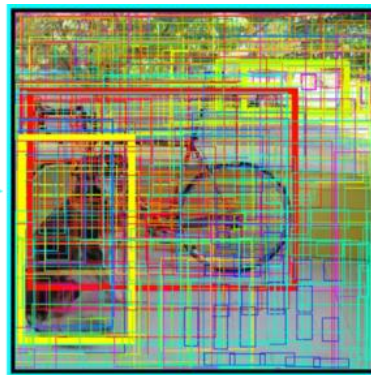
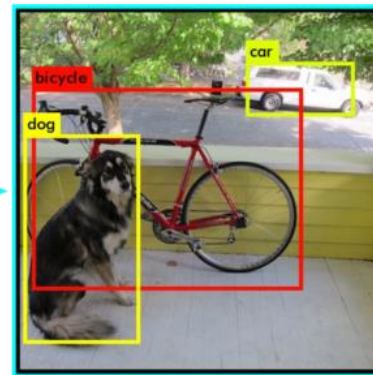


Image grid
 $S \times S$
 $S = 13$



Prediction
Every grid cell B boxes
 $B = 5$

So:
 $13 \times 13 \times 5 = 845$ boxes

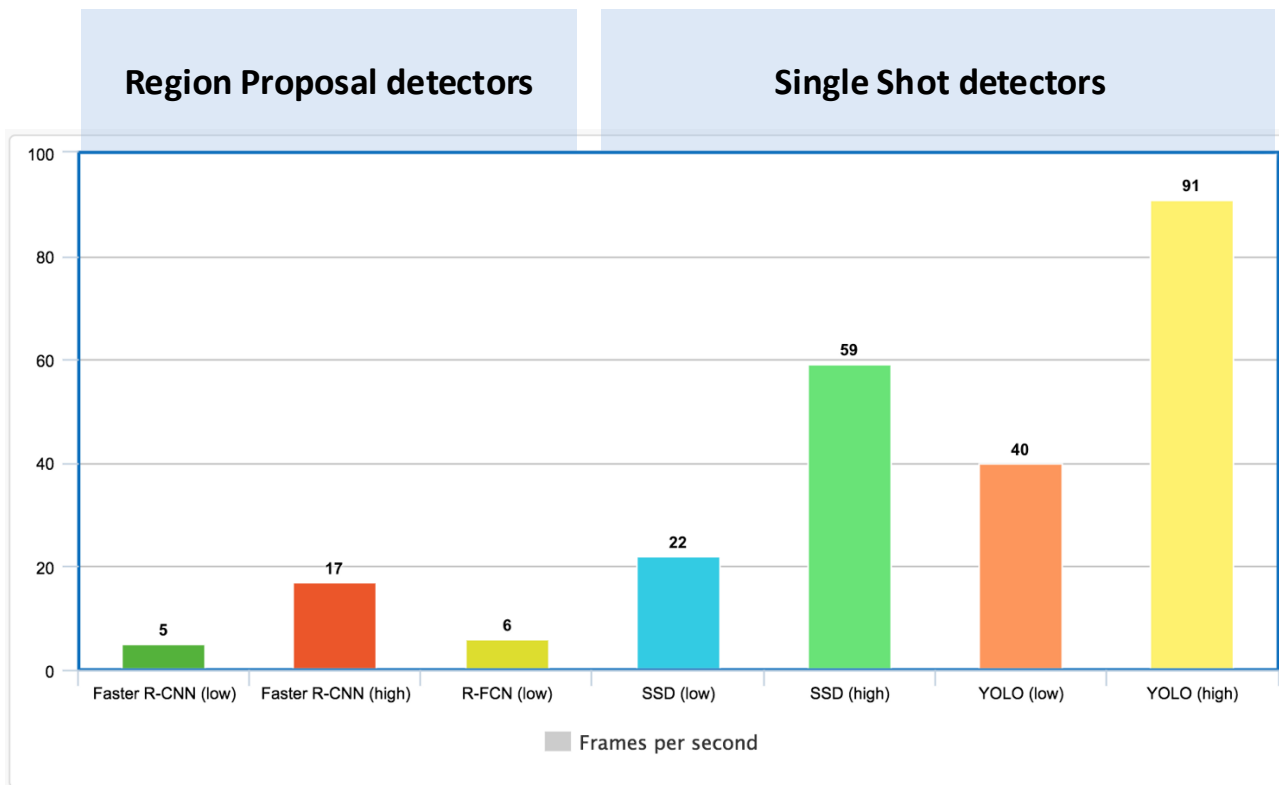


Thresholding:
Box confidence
score > 0.3
Non-Maximum suppression
Class predictions with highest
class confidence score

Single-Shot Detection

SSD / YOLO vs. R-CNNs

Warning:
comparison is
difficult

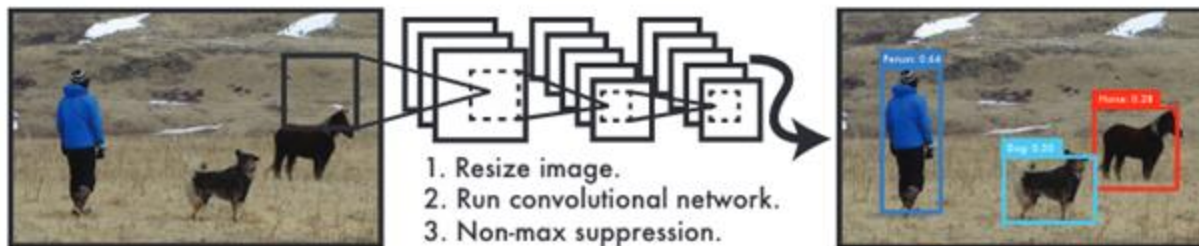


[Source: https://medium.com/@jonathan_hui/object-detection-speed-and-accuracy-comparison-faster-r-cnn-r-fcn-ssd-and-yolo-5425656ae359]

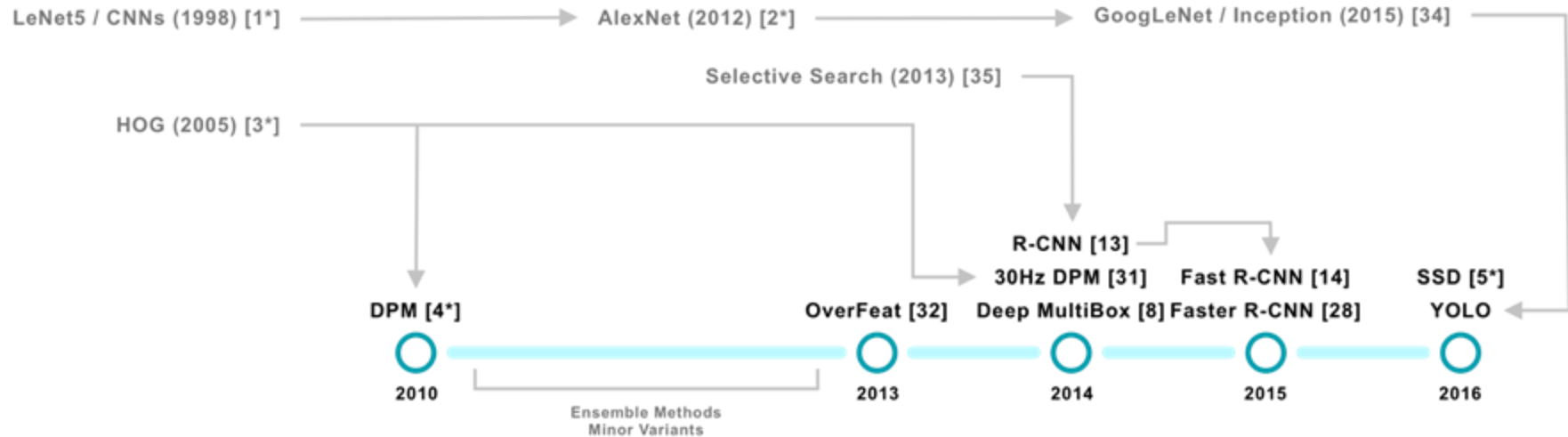
You Only Look Once (YOLO)



- Simple architecture
- Single regression problem, from pixels to bounding box coordinates and class probabilities:
simpler to optimize
- **FAST:** 45 FPS for YOLO and 150 FPS for YOLO-Fast
- Sees the global context, not just patches



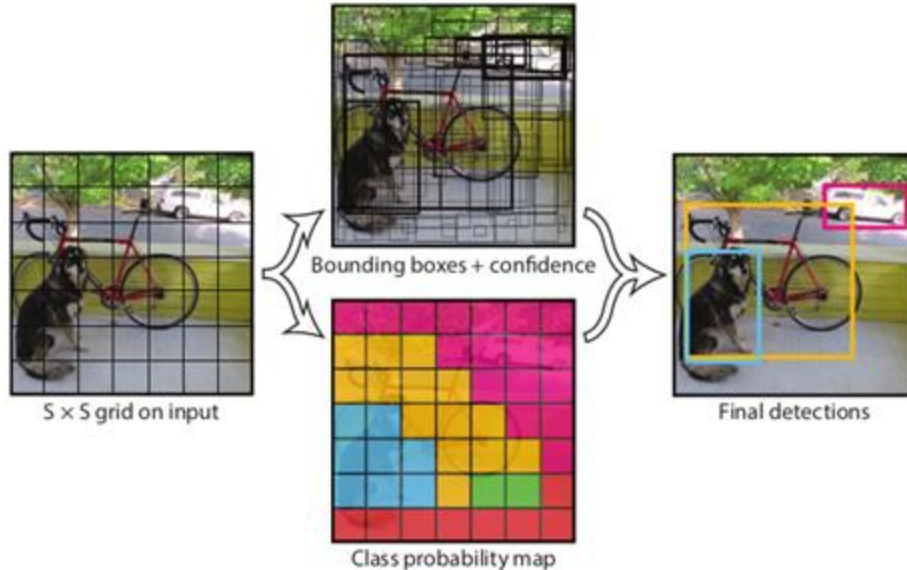
You Only Look Once (YOLO)



Developed and released a novel object detection approach that:

- Is 2.5 times faster than competing methods and 6.5 times faster than SoTA while sacrificing ~10% mAP
- Only consists of a single neural network which is easy to train
- Considers the global context

YOLO – Unified Detection



- Divide image into $S \times S$ grid.
- Each cell predicts:
 - **B** bounding boxes and confidence scores (*Localization*)
 - **C** conditional class probabilities (*Classification*).
for the object whose center it roughly contains.

YOLO - Localization

e.g. for $B=2$



ground truth:

(x,y,w,h)

predicted:

(x_1,y_1,w_1,h_1,c_1)

(x_2,y_2,w_2,h_2,c_2)

B bounding boxes and confidence scores.

$(x,y,w,h,c) \rightarrow B \times 5$ outputs

- x,y : center of the b. box
- w,h : dimensions of b. box
- c :

$$\text{Pr}(\text{Object}) * \text{IOU}_{\text{pred}}^{\text{truth}}$$

YOLO - Localization

e.g. for $B=2$



ground truth:

(x,y,w,h)

predicted:

$(x1,y1,w1,h1,c1)$

$(x2,y2,w2,h2,c2)$

B bounding boxes and confidence scores.

$(x,y,w,h,c) \rightarrow B \times 5$ outputs

- x,y : center of the b. box
- w,h : dimensions of b. box
- c :

$$\text{Pr}(\text{Object}) * \text{IOU}_{\text{pred}}^{\text{truth}}$$

YOLO - Localization

e.g. for $B=2$



ground truth:

predicted:
 $(x1, y1, w1, h1, 0)$

B bounding boxes and confidence scores.

$(x, y, w, h, c) \rightarrow B \times 5$ outputs

- x, y : center of the b. box
- w, h : dimensions of b. box
- c :

$$\text{Pr}(\text{Object}) * \text{IOU}_{\text{pred}}^{\text{truth}}$$

YOLO - Classification



$$(p_{\text{dog}}, p_{\text{bike}}, p_{\text{car}}) = (0.1, 0.2, 0.8)$$

C conditional class probabilities.

$(p_1, \dots, p_C) \rightarrow C$ outputs

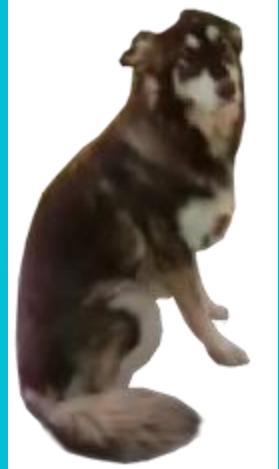
- p_i :

$$\Pr(\text{Class}_i | \text{Object})$$

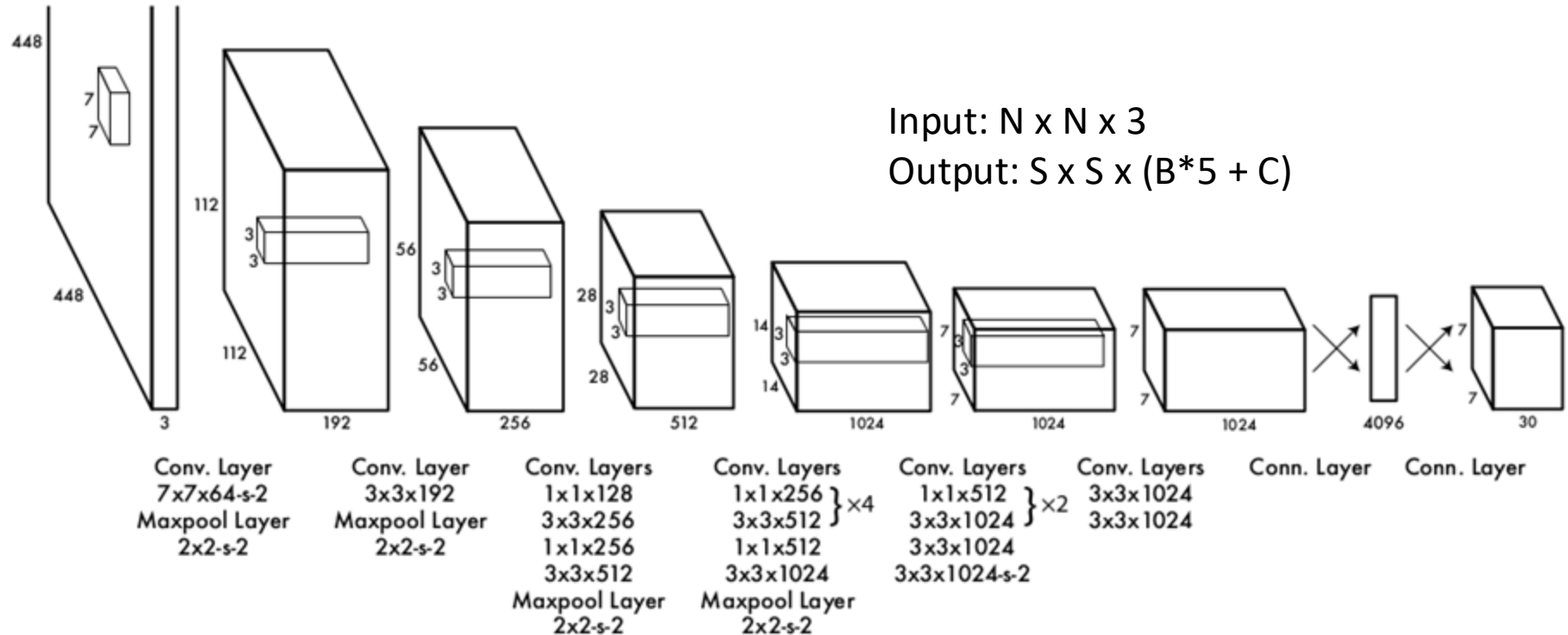
YOLO - Confidence Score

$$\underbrace{\Pr(\text{Class}_i | \text{Object})}_{\text{Classification output}} * \underbrace{\Pr(\text{Object}) * \text{IOU}_{\text{pred}}^{\text{truth}}}_{\text{Localization output}} = \underbrace{\Pr(\text{Class}_i) * \text{IOU}_{\text{pred}}^{\text{truth}}}_{\text{Class-specific confidence score}}$$

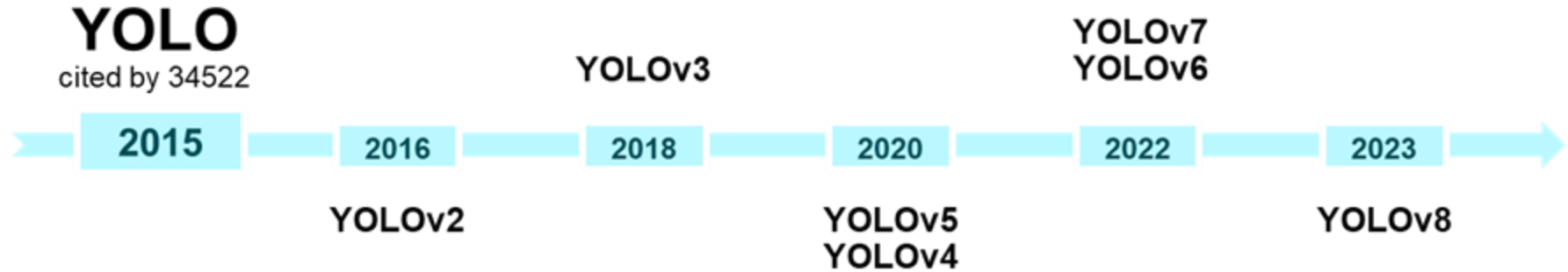
dog, conf=0.93



YOLO - Architecture

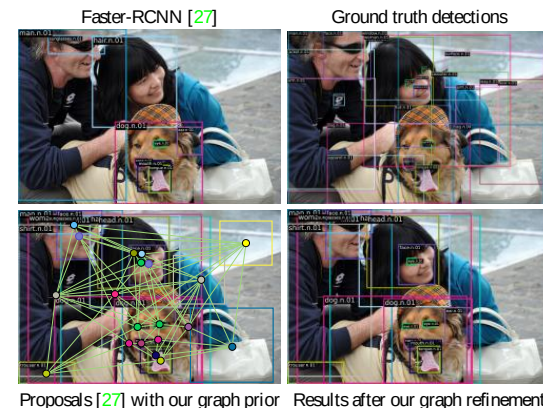
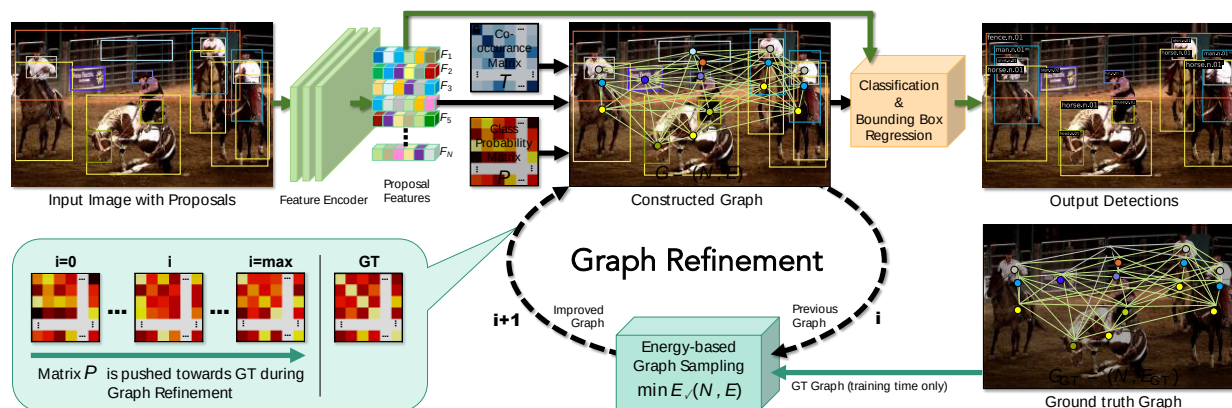
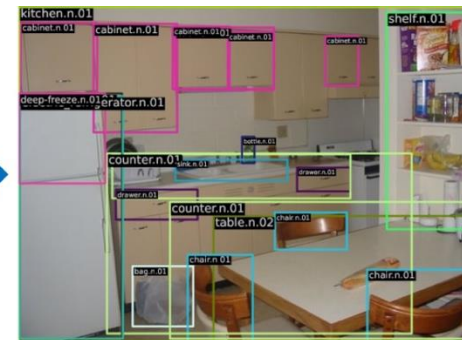


YOLO - Architecture



Recent Work: Object Detection

- Idea: Leverage context information about surrounding objects
- Context encoded via graph network of object proposals
- Learn & use co-occurrence statistics



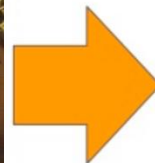
Today's Agenda

- Learning region proposals - Faster R-CNN
- Feature Pyramid Network (FPN)
- Single Shot Detectors (SSD) / Yolo
- **Transfer learning**
- Network architectures
 - ResNet
 - U-Net
 - Autoencoders

Classification Model Size

ImageNet Challenge: 2014

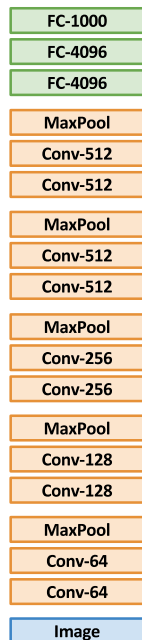
AlexNet



- Model size steadily increases
- Expensive training on image net
- Idea: Re-use training results

Transfer Learning

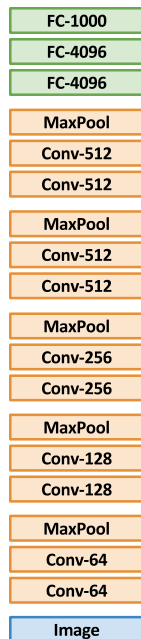
1. Train on ImageNet



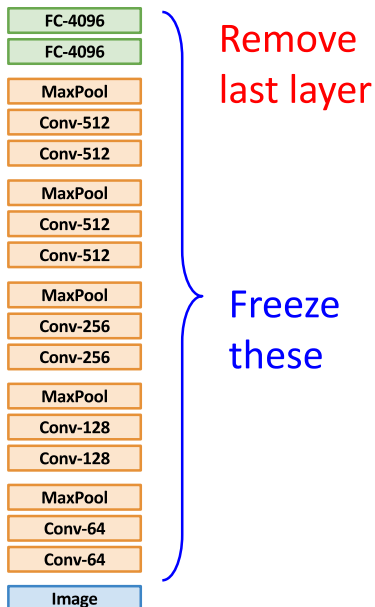
Transfer Learning

Transfer Learning: Feature Extraction

1. Train on ImageNet



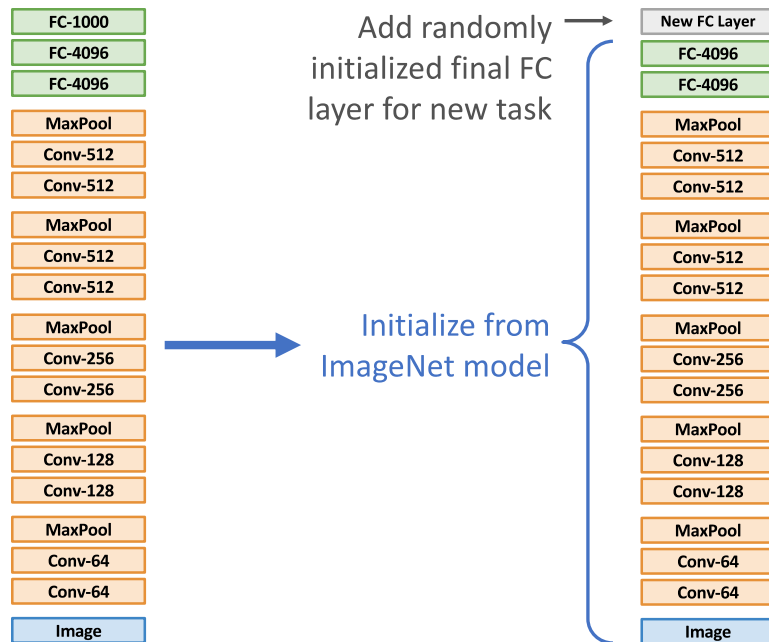
2. Extract features with CNN, train linear model



Transfer Learning

Transfer Learning: Fine-Tuning

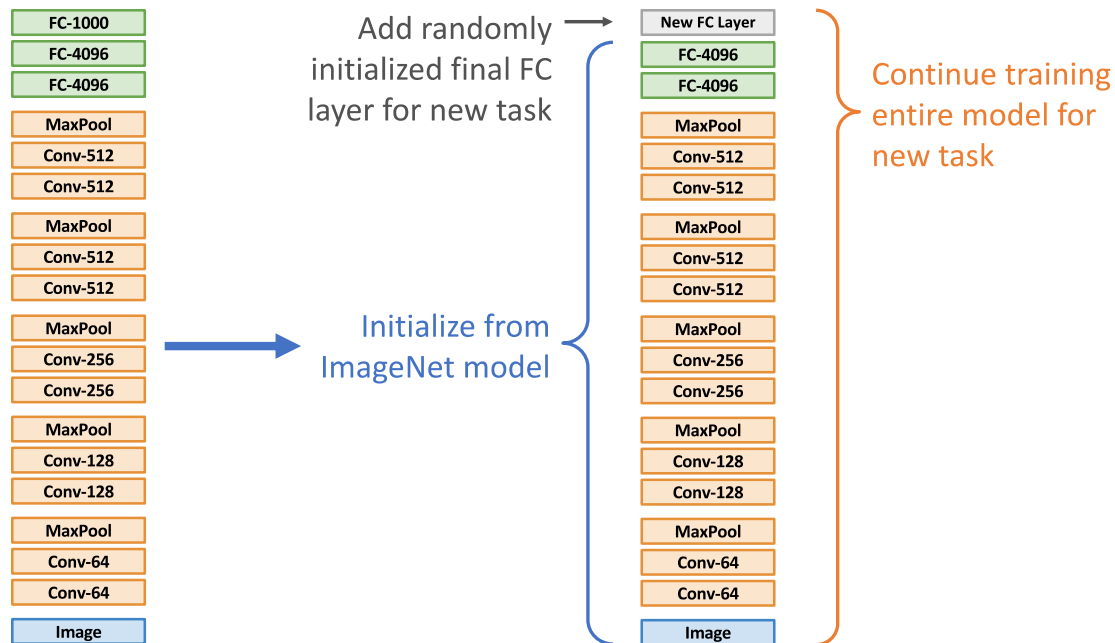
1. Train on ImageNet



Transfer Learning

Transfer Learning: Fine-Tuning

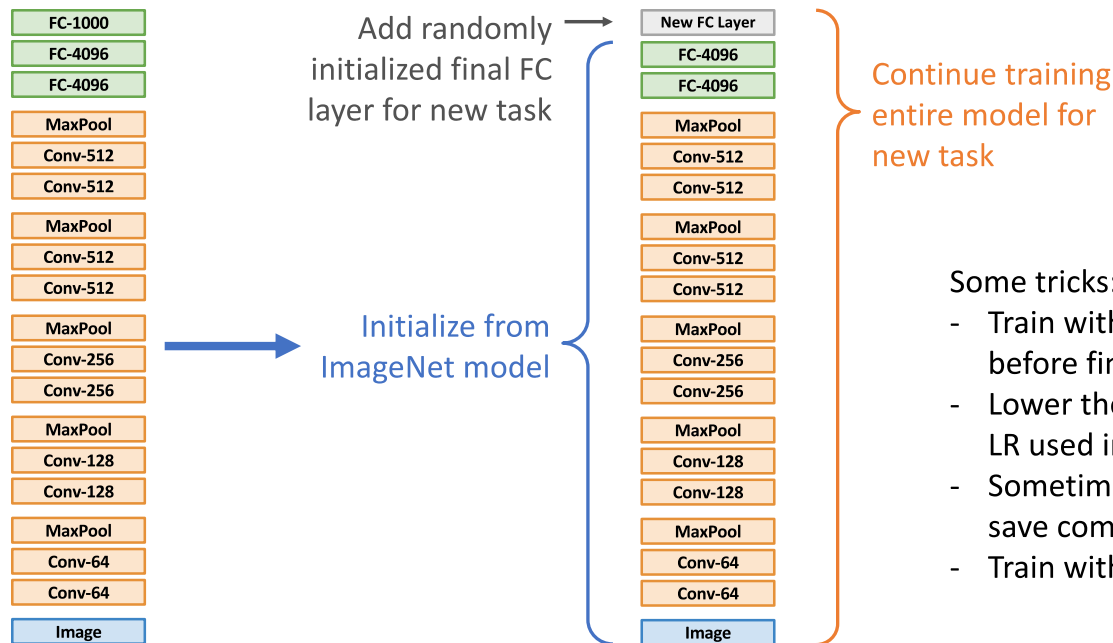
1. Train on ImageNet



Transfer Learning

Transfer Learning: Fine-Tuning

1. Train on ImageNet



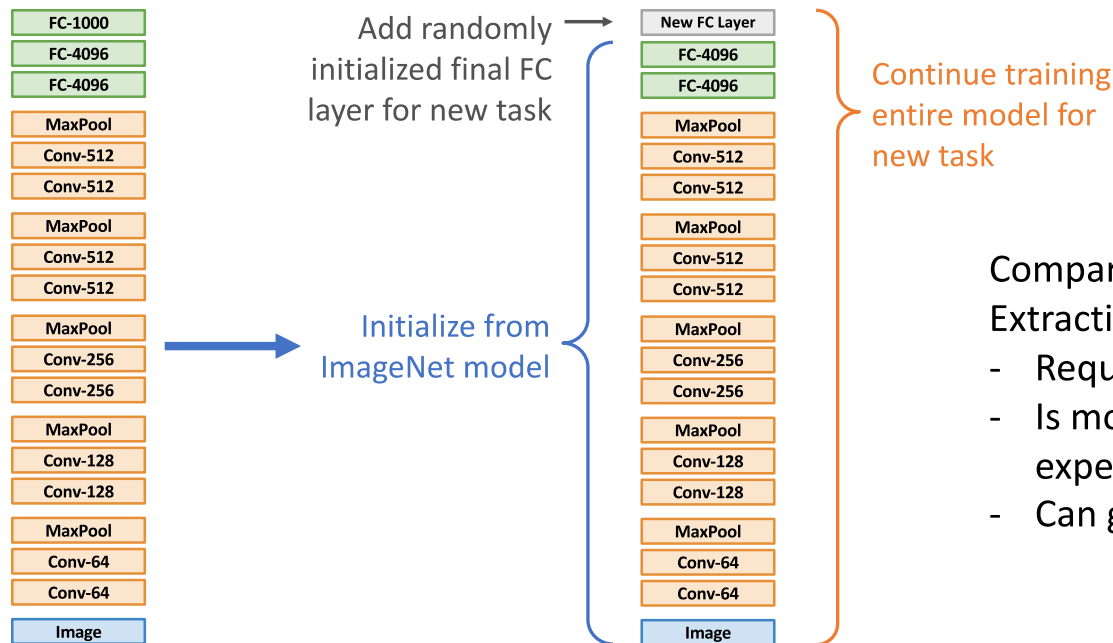
Some tricks:

- Train with feature extraction first before fine-tuning
- Lower the learning rate: use $\sim 1/10$ of LR used in original training
- Sometimes freeze lower layers to save computation
- Train with BatchNorm in "test" mode

Transfer Learning

Transfer Learning: Fine-Tuning

1. Train on ImageNet

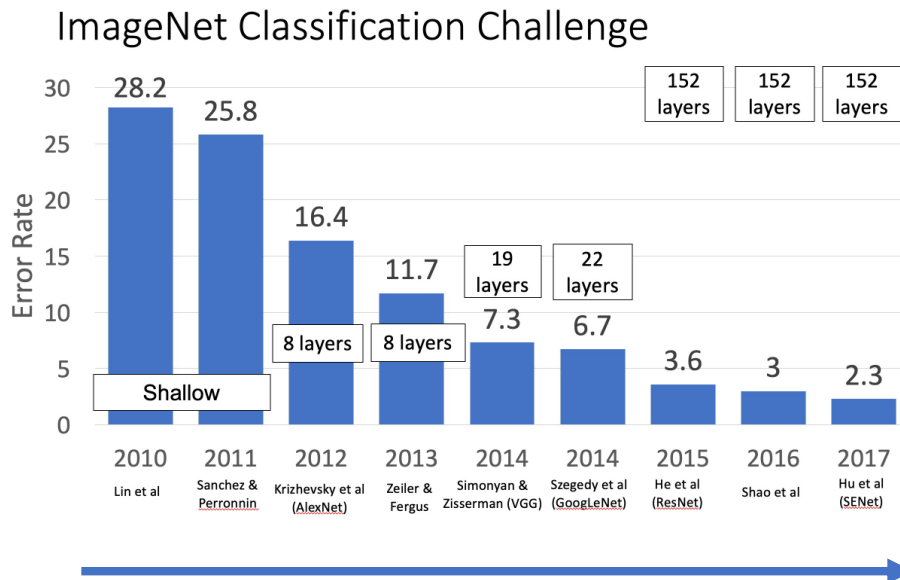


Compared with Feature Extraction, Fine-Tuning:

- Requires more data
- Is more computationally expensive
- Can give higher accuracies

Transfer Learning

Transfer Learning: Architecture Matters!

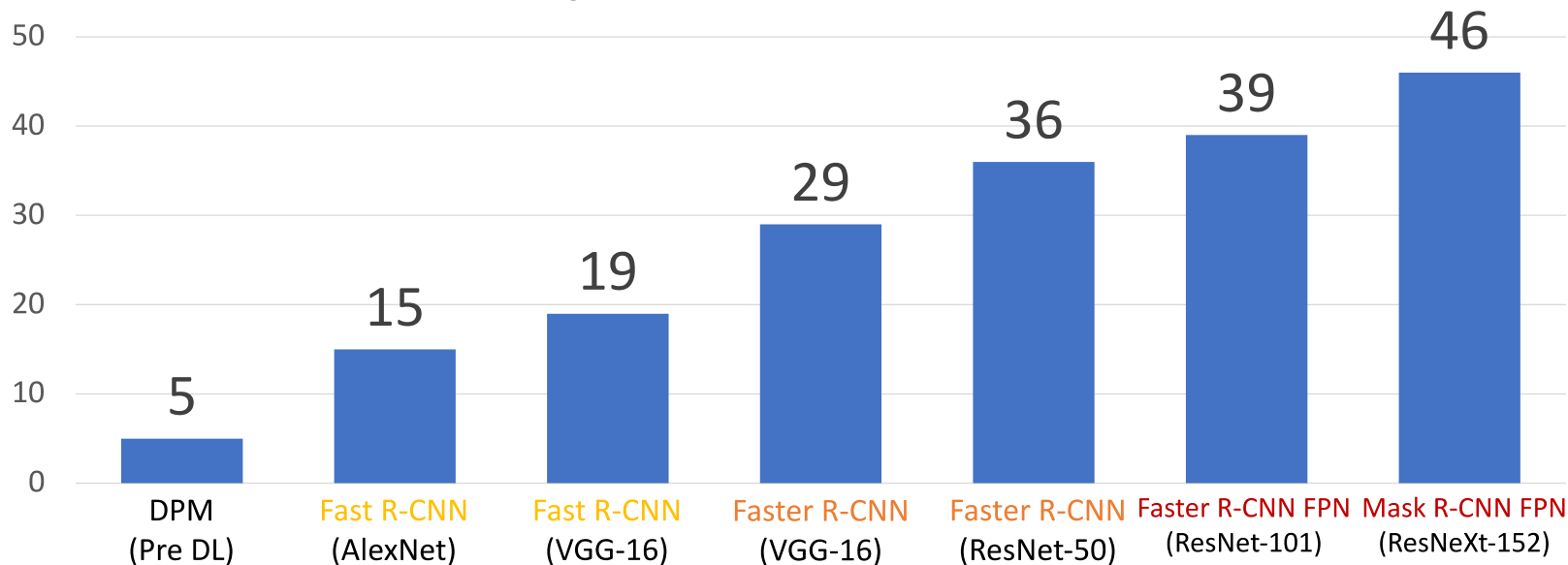


Improvements in CNN architectures lead to improvements in many downstream tasks thanks to transfer learning!

Transfer Learning

Transfer Learning: Architecture Matters!

Object Detection on COCO

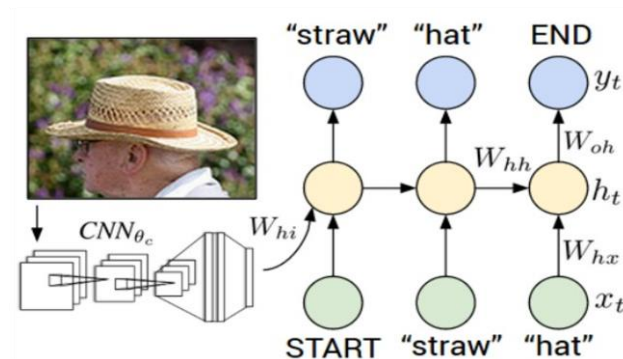
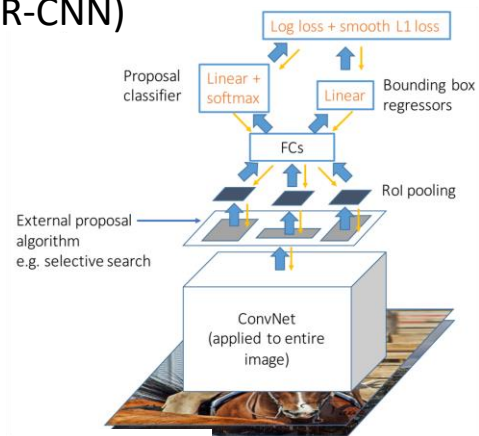


Ross Girshick, "The Generalized R-CNN Framework for Object Detection", ICCV 2017 Tutorial on Instance-Level Visual Recognition

Transfer Learning

Transfer learning is pervasive!
It's the norm, not the exception

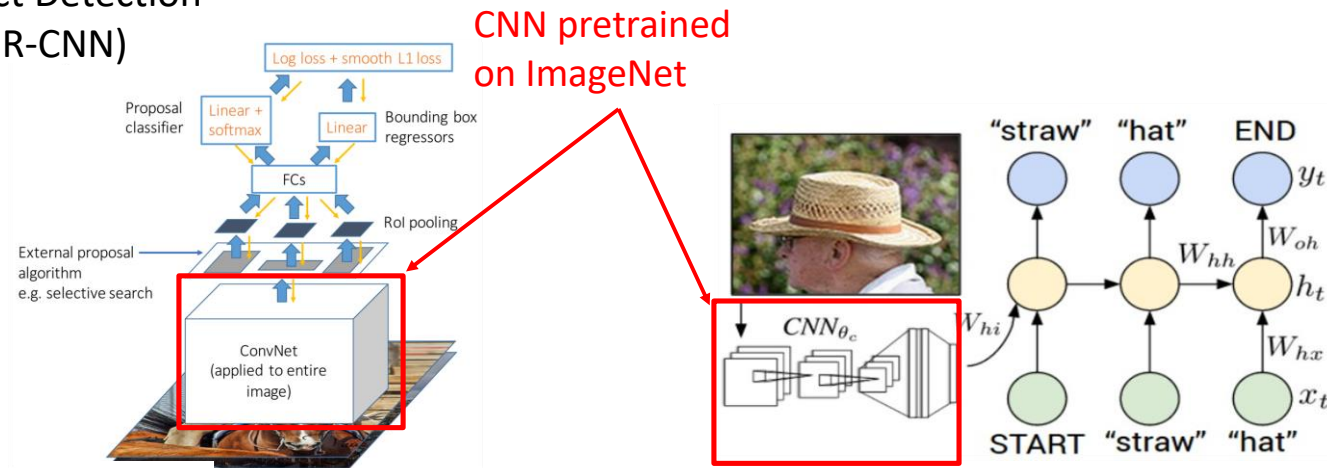
Object Detection (Fast R-CNN)



Transfer Learning

Transfer learning is pervasive!
It's the norm, not the exception

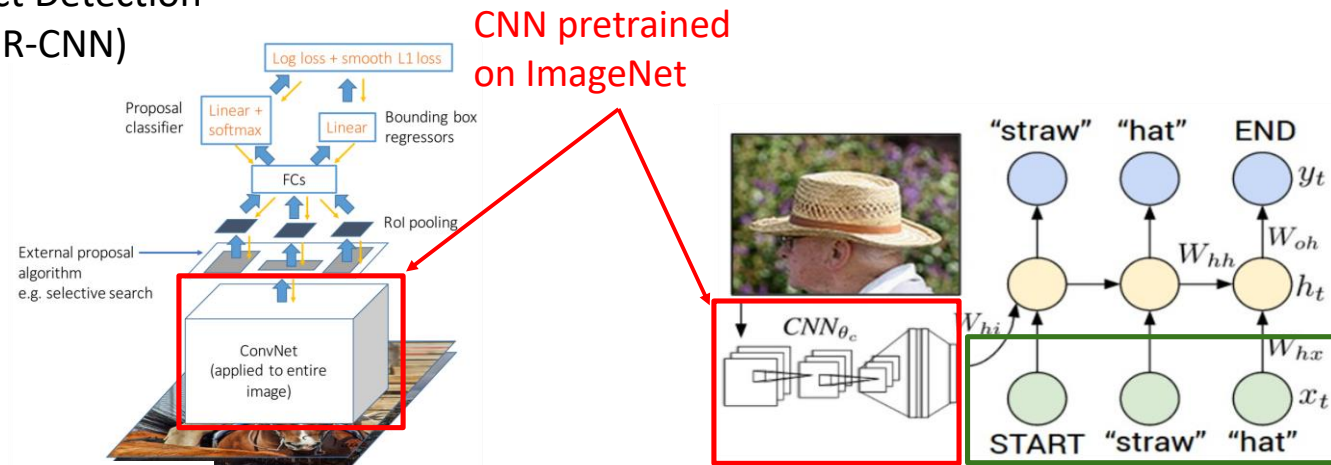
Object Detection (Fast R-CNN)



Transfer Learning

Transfer learning is pervasive!
It's the norm, not the exception

Object Detection (Fast R-CNN)



Girshick, "Fast R-CNN", ICCV 2015
Figure copyright Ross Girshick, 2015. Reproduced with permission.

Karpathy and Fei-Fei, "Deep Visual-Semantic Alignments
for Generating Image Descriptions", CVPR 2015

Today's Agenda

- Learning region proposals - Faster R-CNN
- Feature Pyramid Network (FPN)
- Single Shot Detectors (SSD) / Yolo
- Transfer learning
- Network architectures
 - ResNet
 - U-Net
 - Autoencoders

ResNet (ILSVRC 2015 winner)

AlexNet, 8 layers
(ILSVRC 2012)



VGG, 19 layers
(ILSVRC 2014)

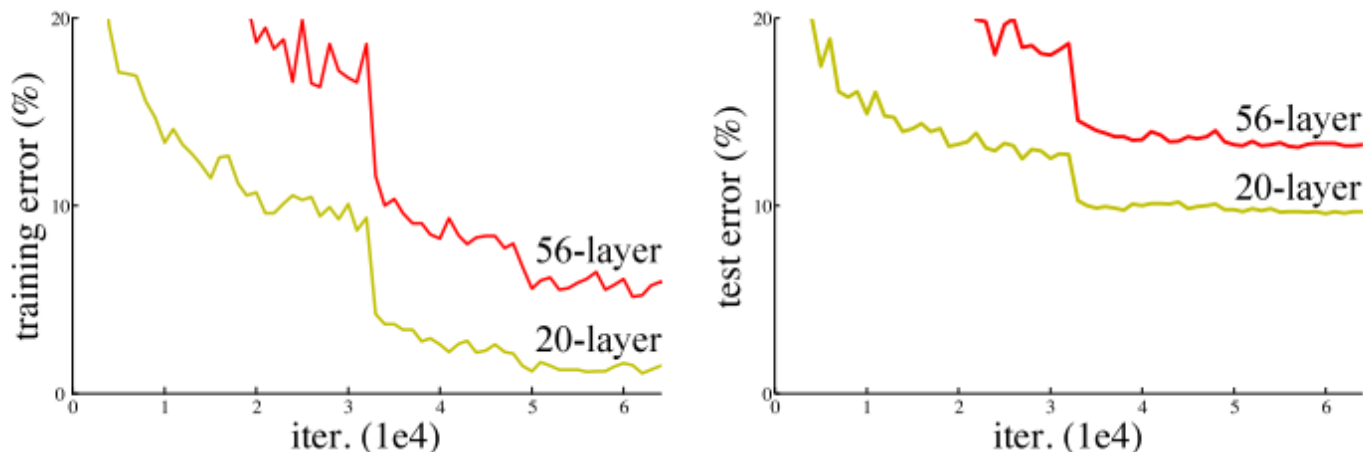


ResNet, 152 layers
(ILSVRC 2015)



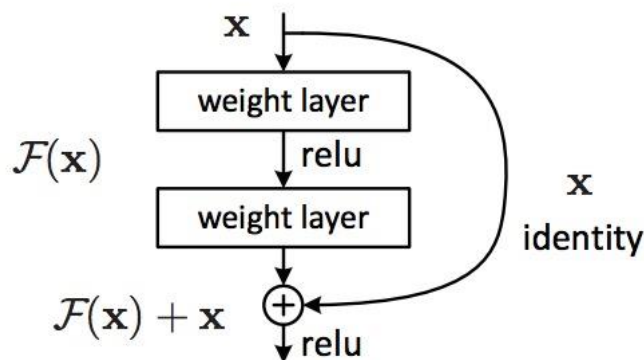
ResNet – Motivation: Vanishing Gradients

Deeper networks tend to saturate. Problem comes from vanishing gradients



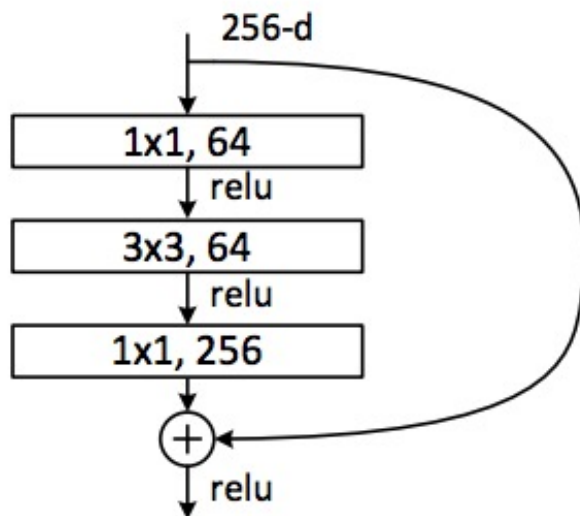
ResNet (ILSVRC 2015 winner)

- The residual module
 - Introduce *skip* or *shortcut* connections (existing before in various forms in literature)
 - Make it easy for network layers to represent the identity mapping



ResNet (ILSVRC 2015 winner)

Deeper residual module
(bottleneck)



- Directly performing 3×3 convolutions with 256 feature maps at input and output:
 $256 \times 256 \times 3 \times 3 \approx 600K$ operations
- Using 1×1 convolutions to reduce 256 to 64 feature maps, followed by 3×3 convolutions, followed by 1×1 convolutions to expand back to 256 maps:
 $256 \times 64 \times 1 \times 1 \approx 16K$
 $64 \times 64 \times 3 \times 3 \approx 36K$
 $64 \times 256 \times 1 \times 1 \approx 16K$
Total $\approx 70K$

ResNet (ILSVRC 2015 winner)

- ResNets are collections of many paths of different length, and shorter paths predominantly contribute to training

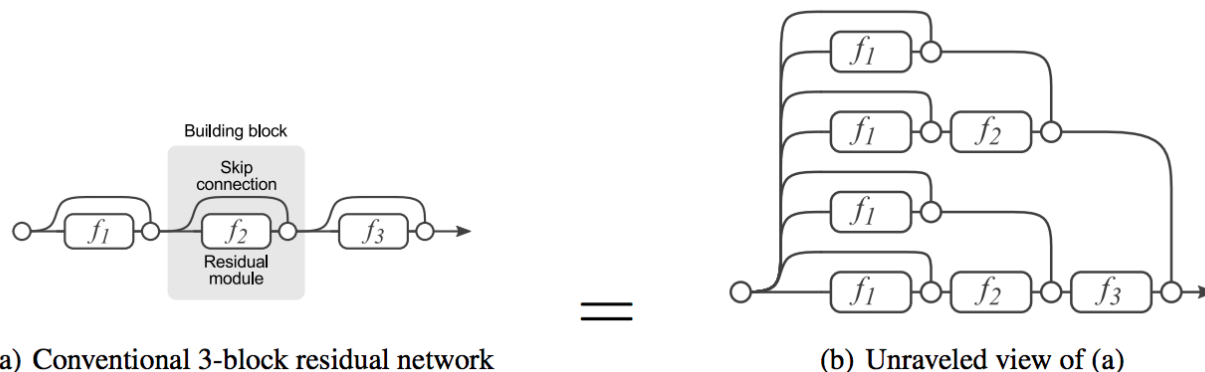
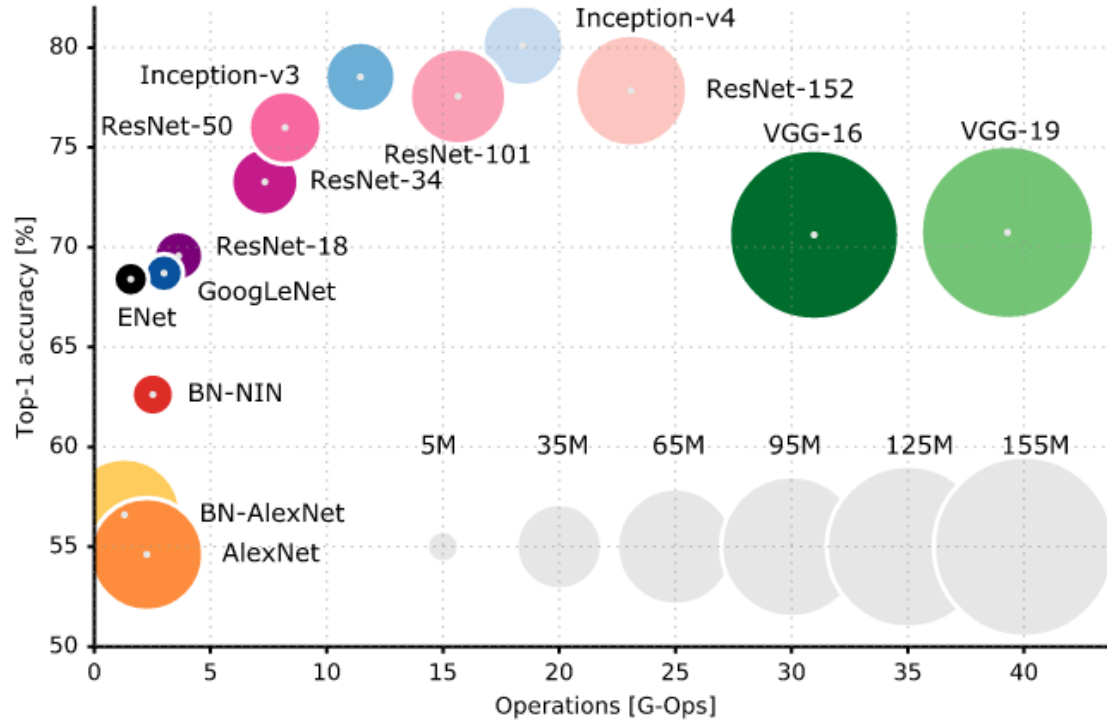


Figure 1: Residual Networks are conventionally shown as (a), which is a natural representation of Equation (1). When we expand this formulation to Equation (6), we obtain an *unraveled view* of a 3-block residual network (b). Circular nodes represent additions. From this view, it is apparent that residual networks have $O(2^n)$ implicit paths connecting input and output and that adding a block doubles the number of paths.

Comparing Architectures



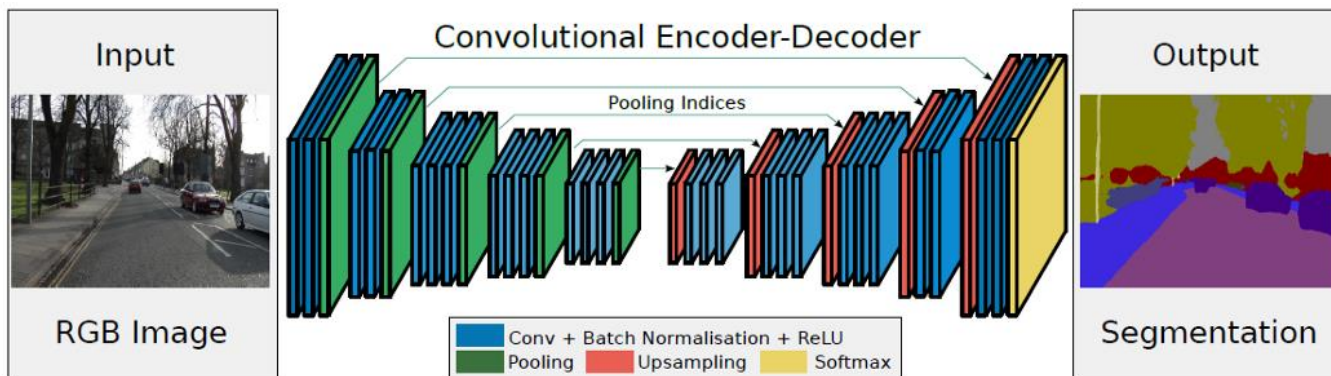
Encoder-Decoder Architecture

Two common use-cases:

- Pixel wise predictions - output is of same dimension of input
 - semantic segmentation (Segnet)
- Auto-encoder for
 - unsupervised feature learning
 - unsupervised pre-training

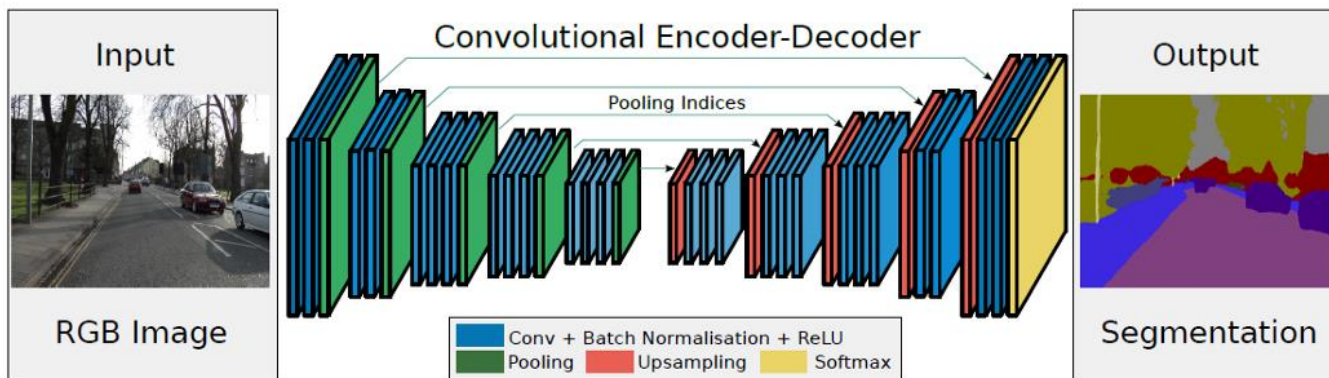
Encoder-Decoder Architecture - SegNet

Segnet uses an encoder-decoder architecture for semantic segmentation.



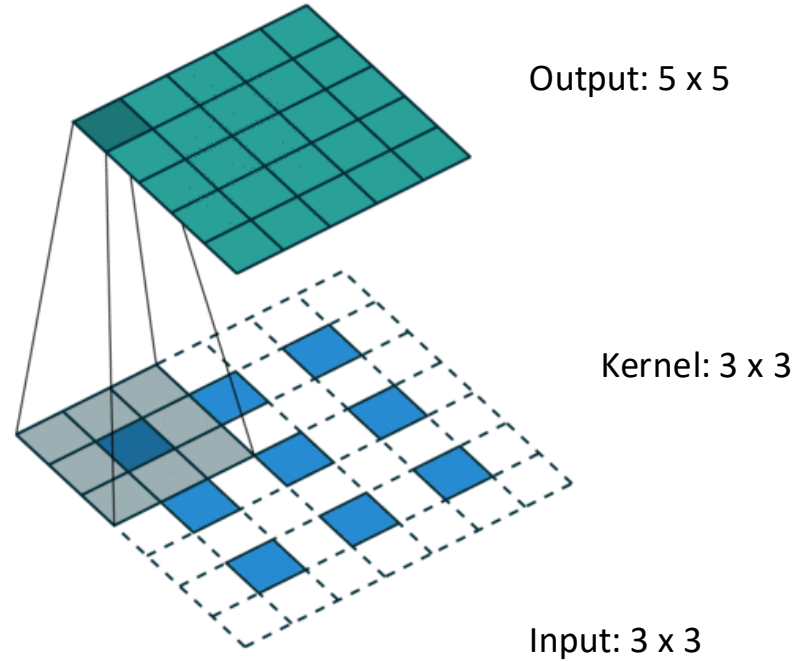
Encoder-Decoder Architecture - SegNet

- Encoder is 'normal' convolution + pooling.
- Decoder uses upsampling + convolution
- (Learned) convolution is to refine the upsampling



SegNet: Transposed Convolution

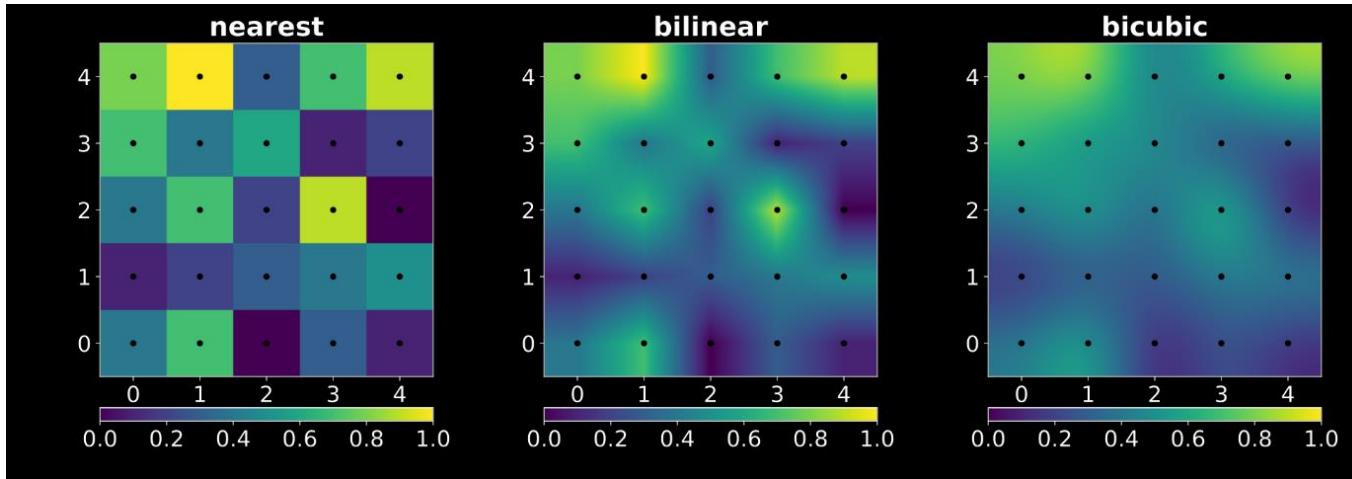
- upsampling: unpooling
- convolution



also: up-convolution,
not deconvolution

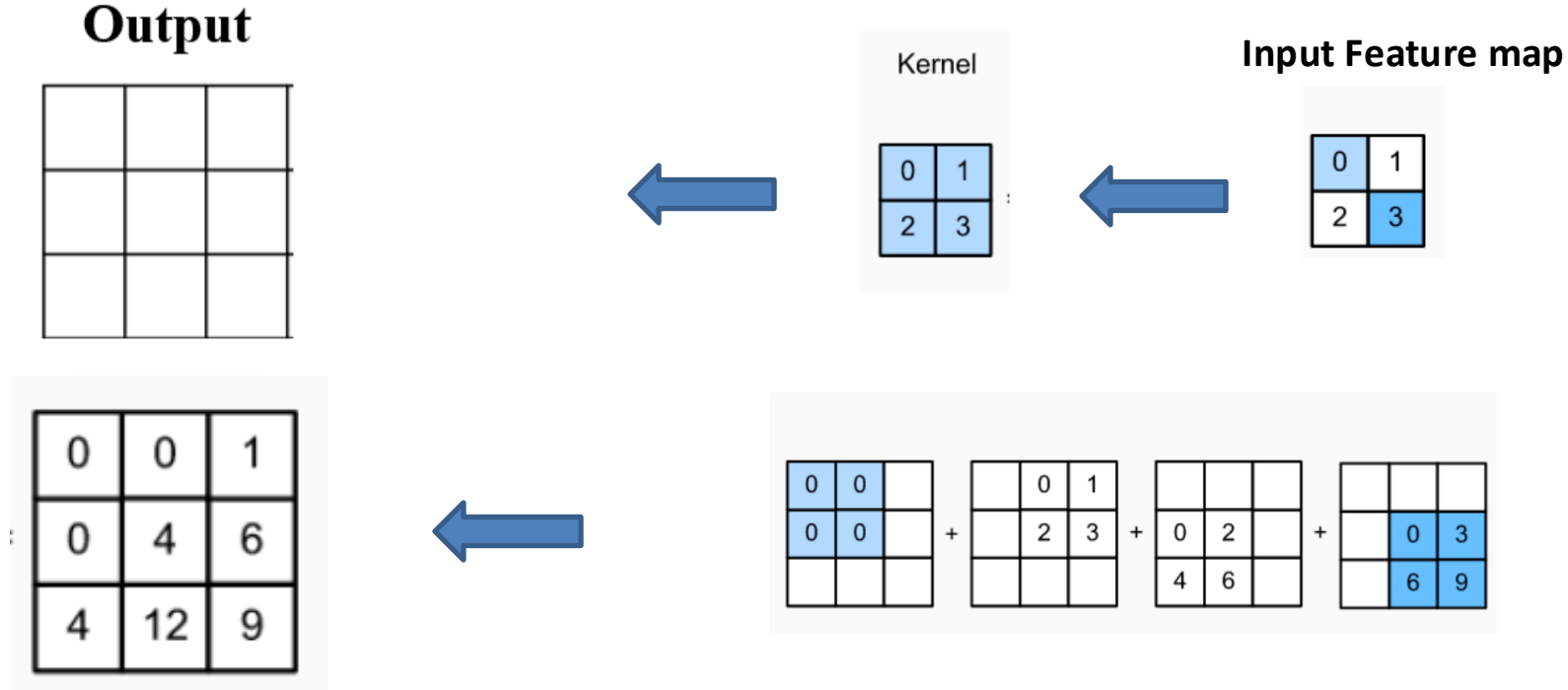
Upsampling Methods

Recall: as alternative for convolution, different interpolation methods exist.



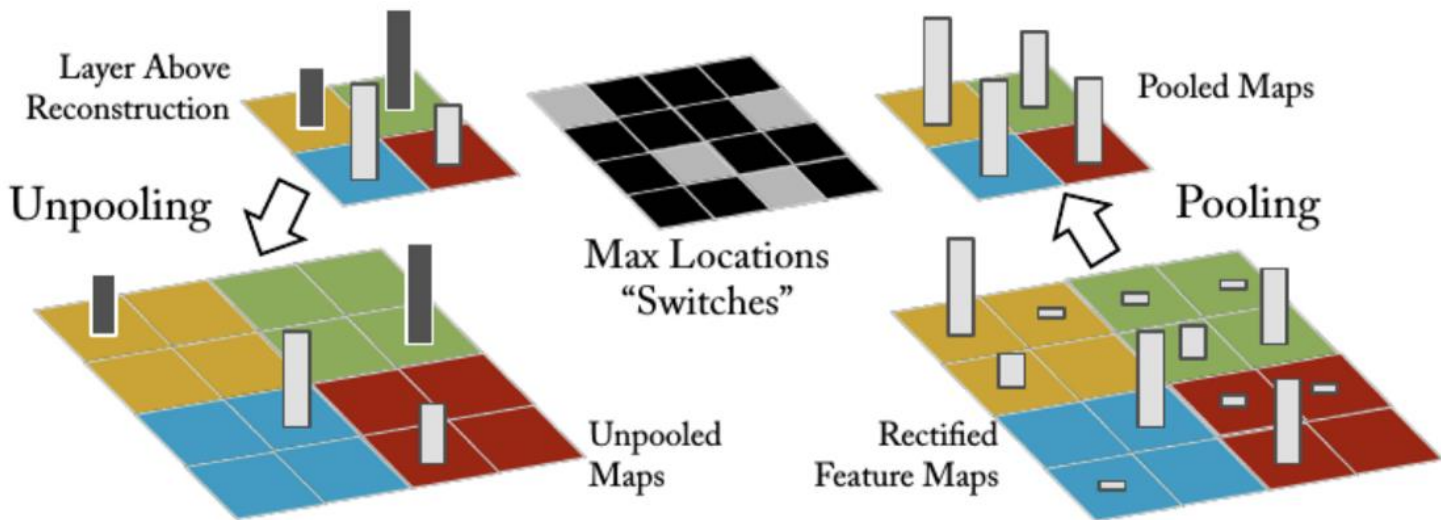
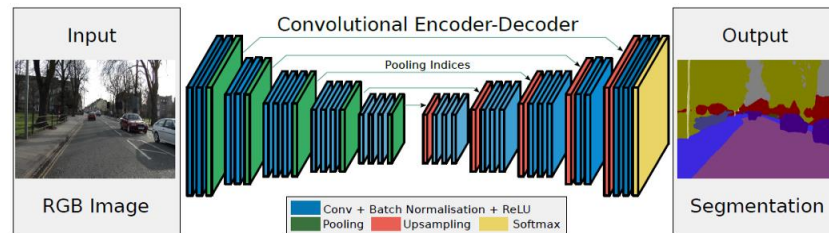
Upsampling Methods

Use learned network – kernels and feature map to ‘deconvolve’ to original pixel space.

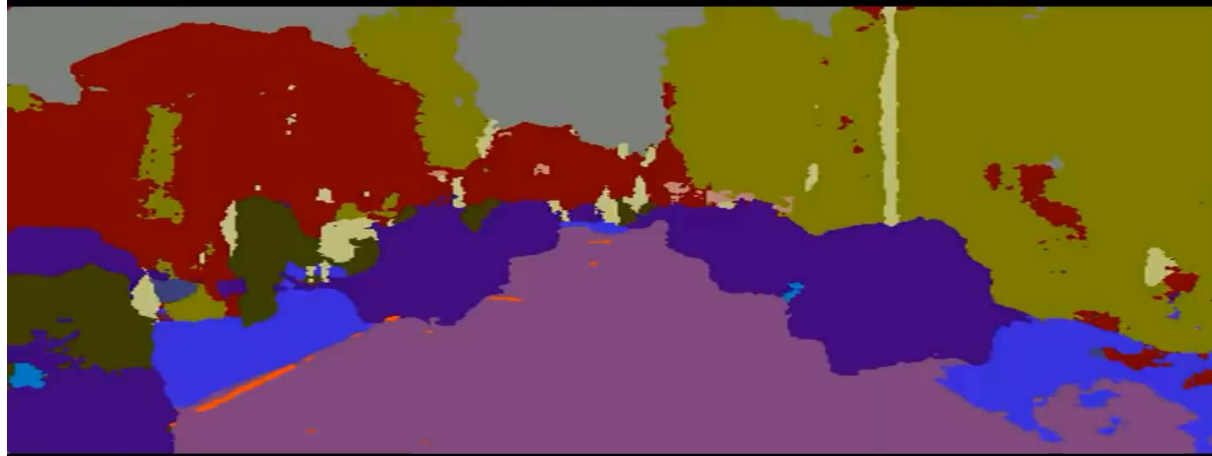


Segnet: Alternative Unpooling operator

The unpooling uses the max-locations ('switches') that have been stored during pooling.

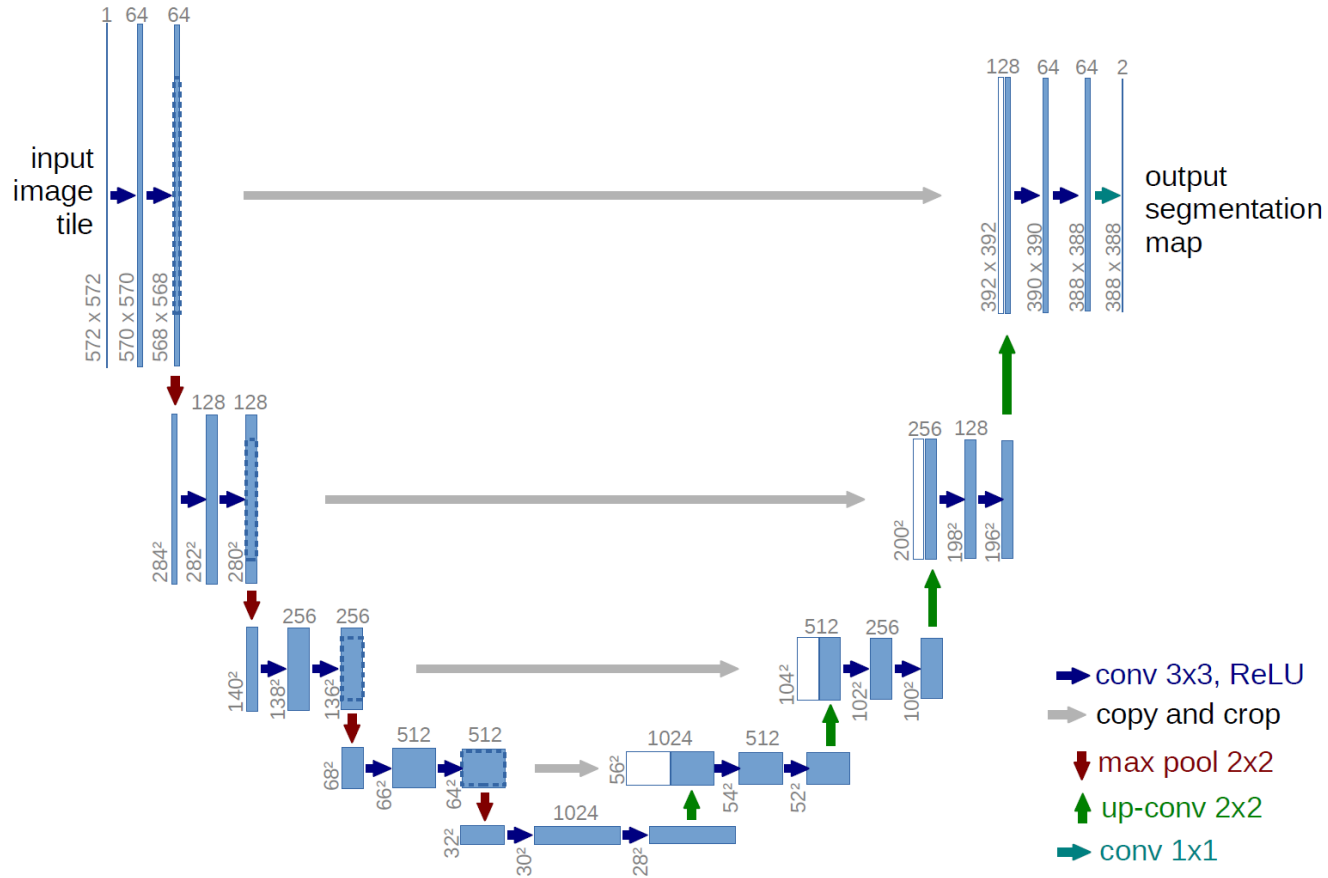


Segnet - Results

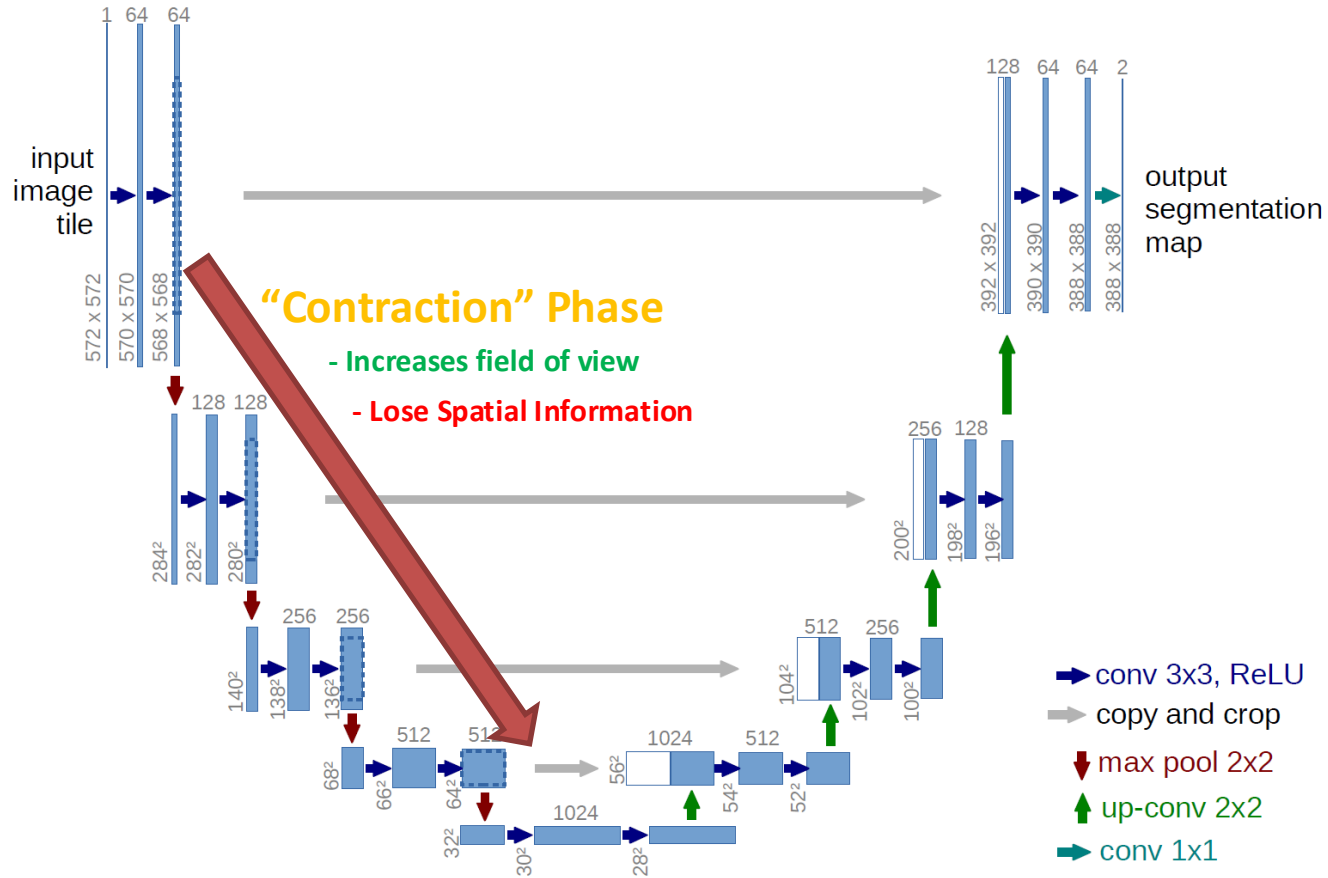


- Sky
- Building
- Pole
- Road Marking
- Road
- Pavement
- Tree
- Sign Symbol
- Fence
- Vehicle
- Pedestrian
- Bike

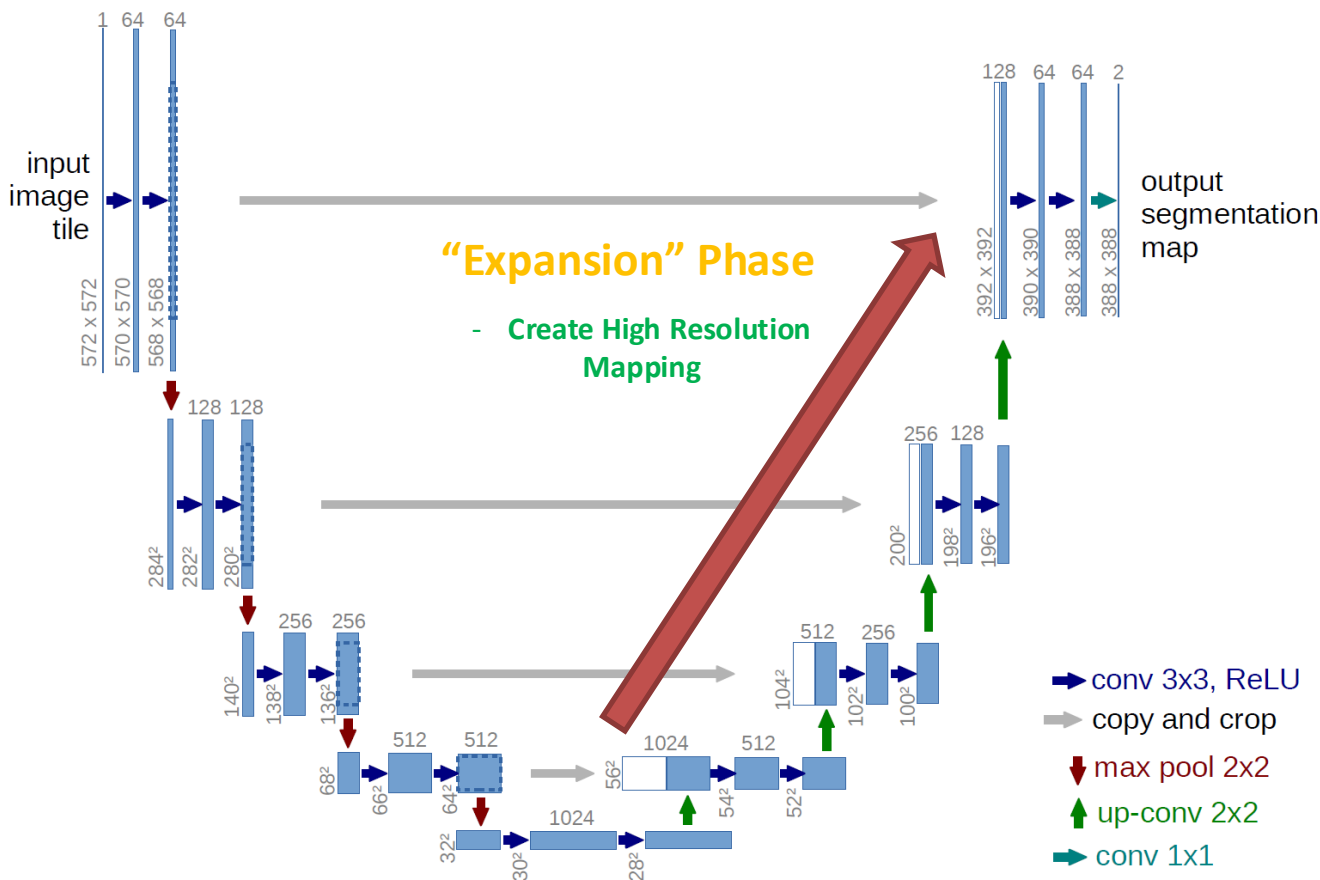
U-Net Architecture



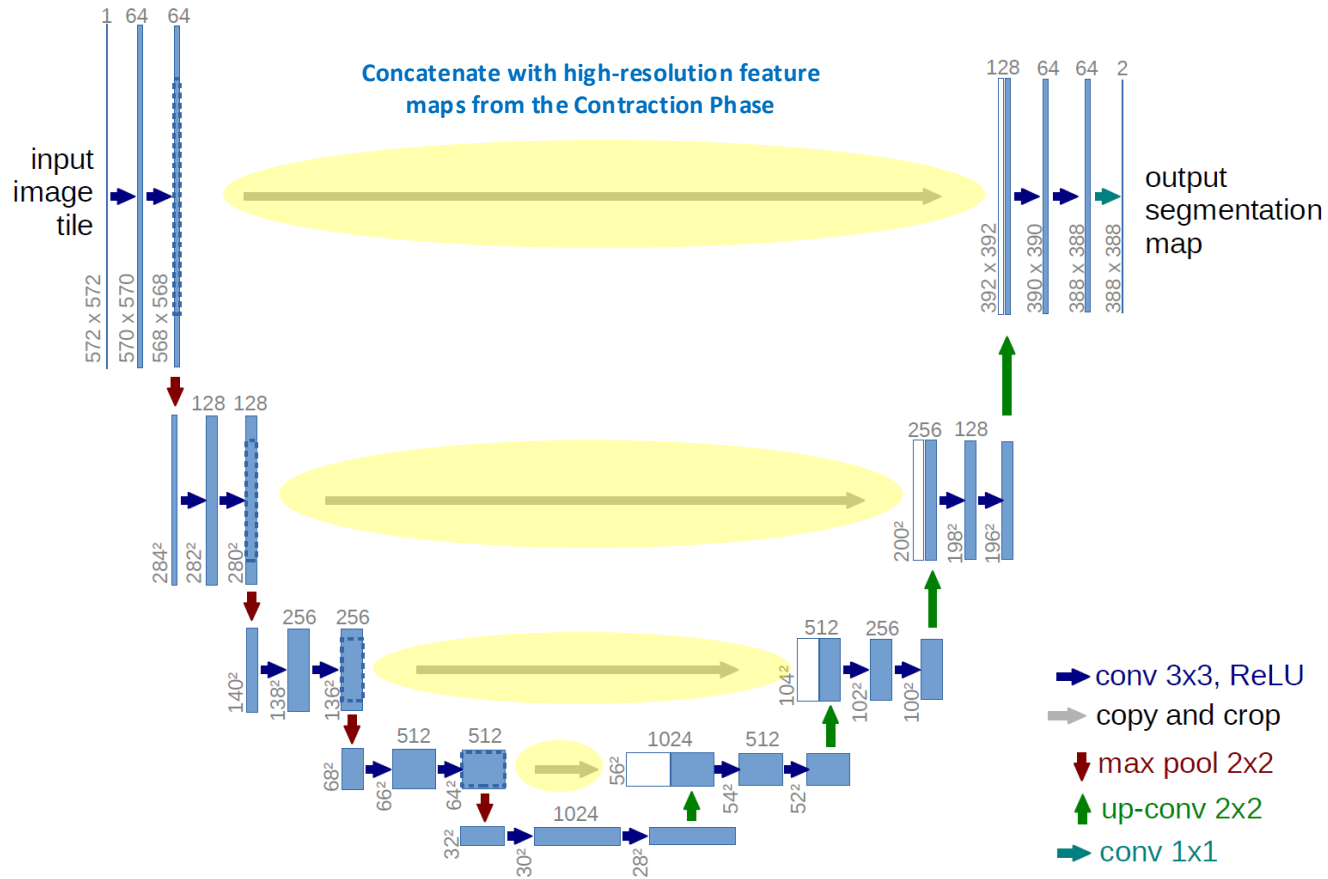
U-Net Architecture



U-Net Architecture



U-Net Architecture



Autoencoders

So far:

Supervised learning:

- training with large number of labeled images

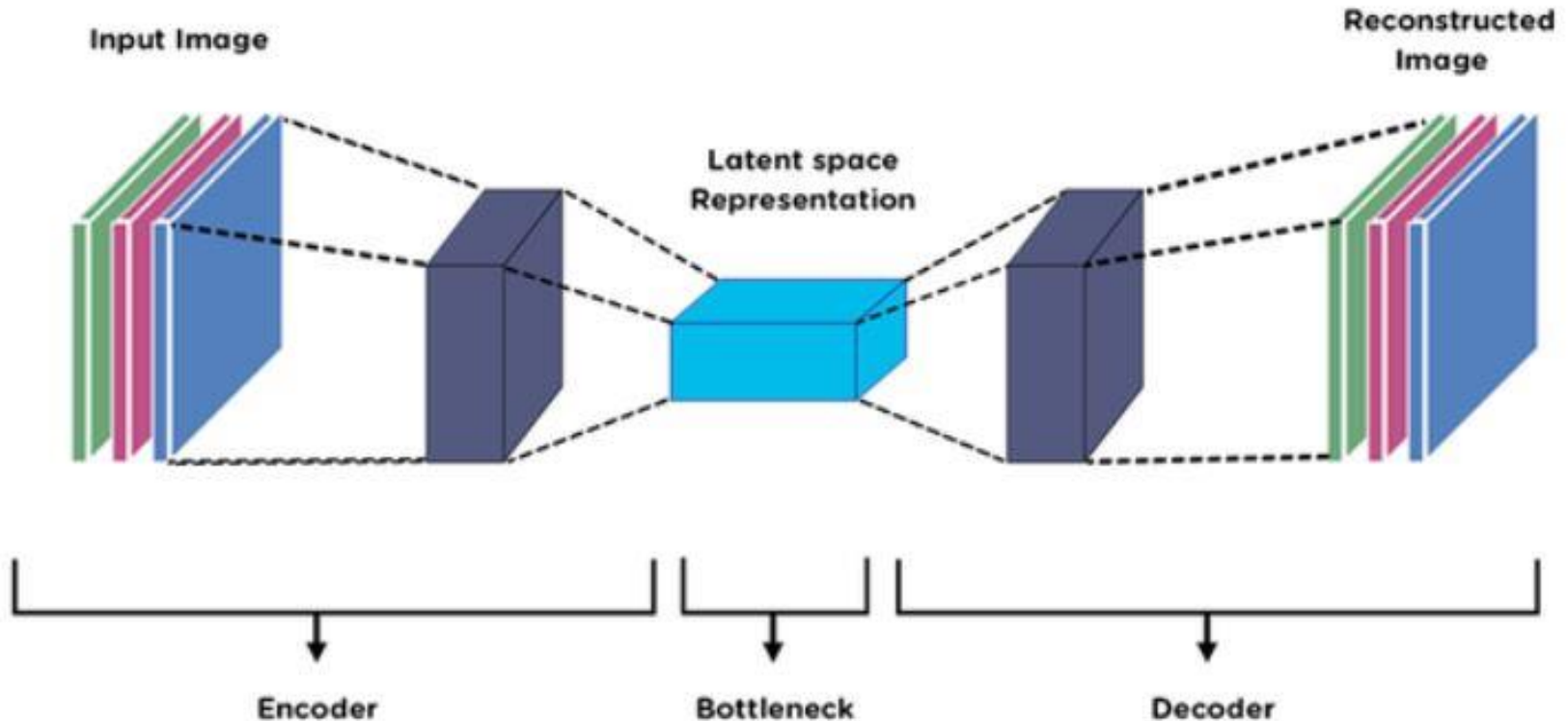
Now:

Unsupervised approach:

- to learn lower dimensional feature representations from unlabeled training data.

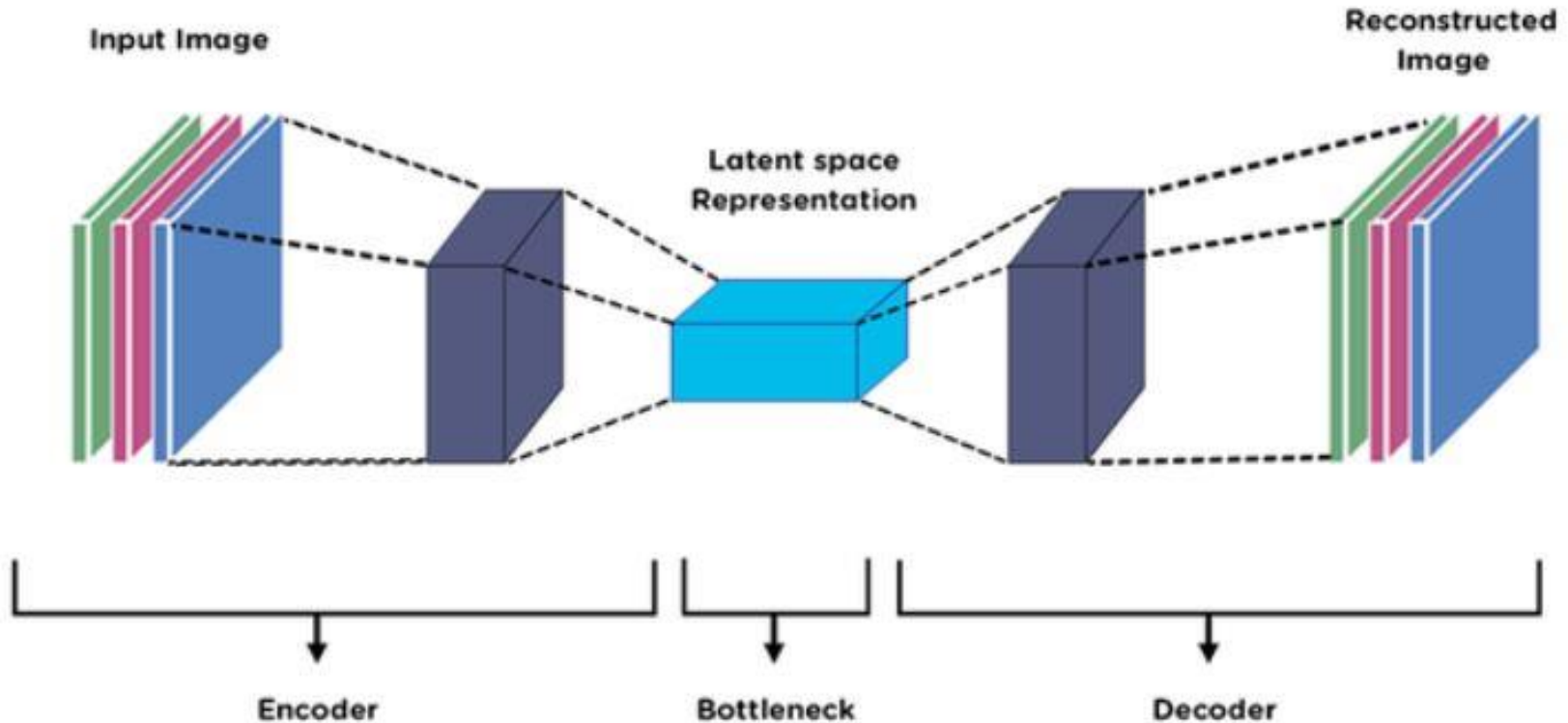
Autoencoders

Encoder – decoder. Network is trained to minimize difference between Reconstructed Image and Input Image (e.g. LSE loss).



Autoencoders

Latent space is of lower dimension than input.
So reconstruction typical not perfect.

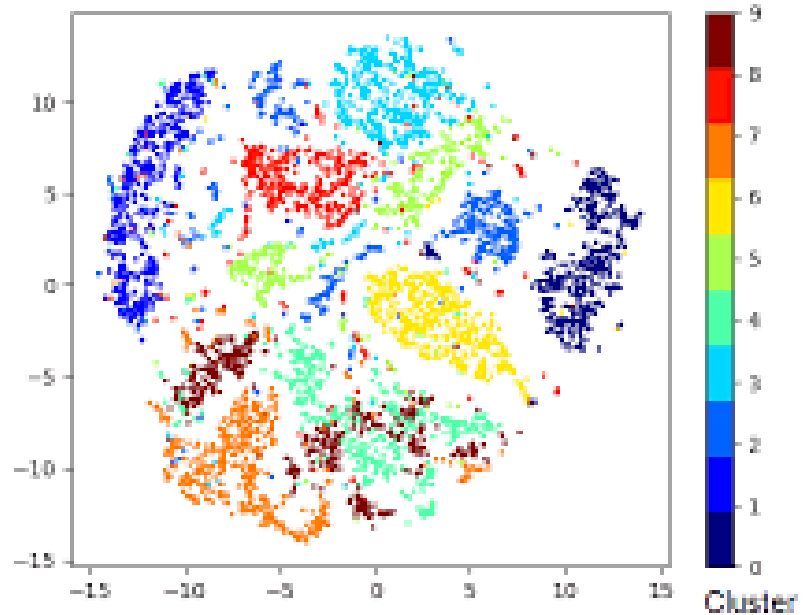


Autoencoders – Use case: Clustering

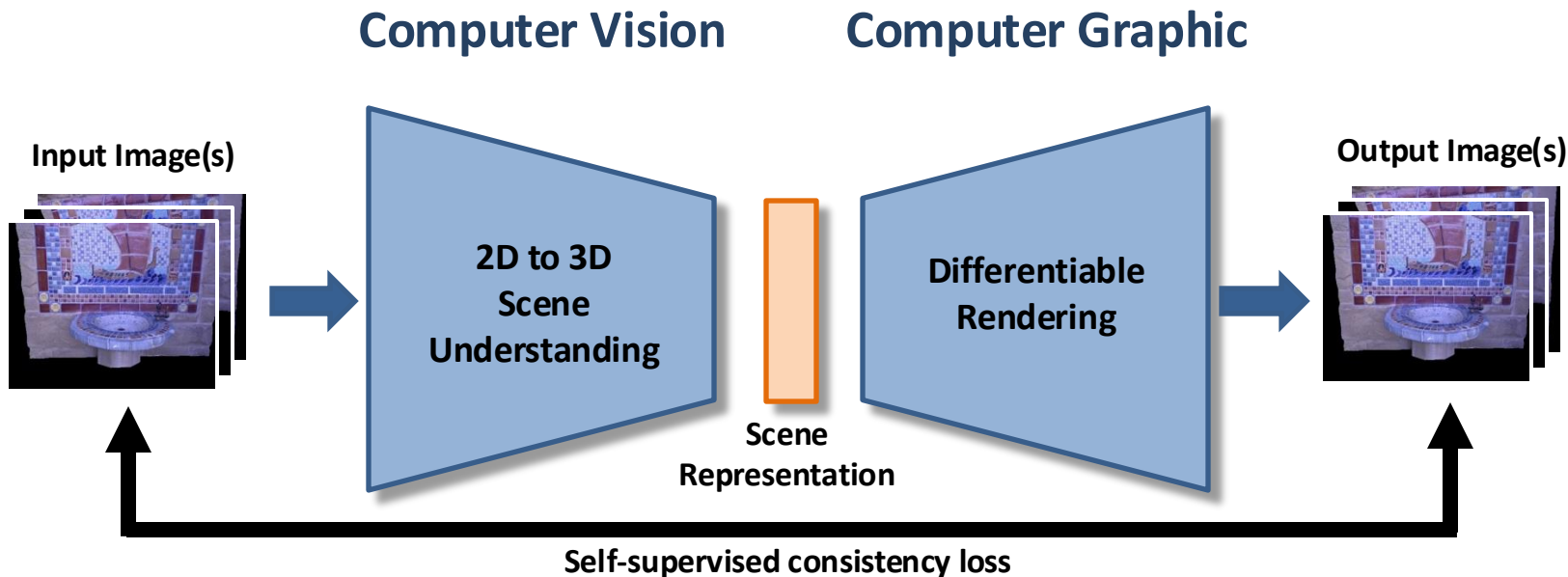
Embedding of MNIST dataset



MNIST

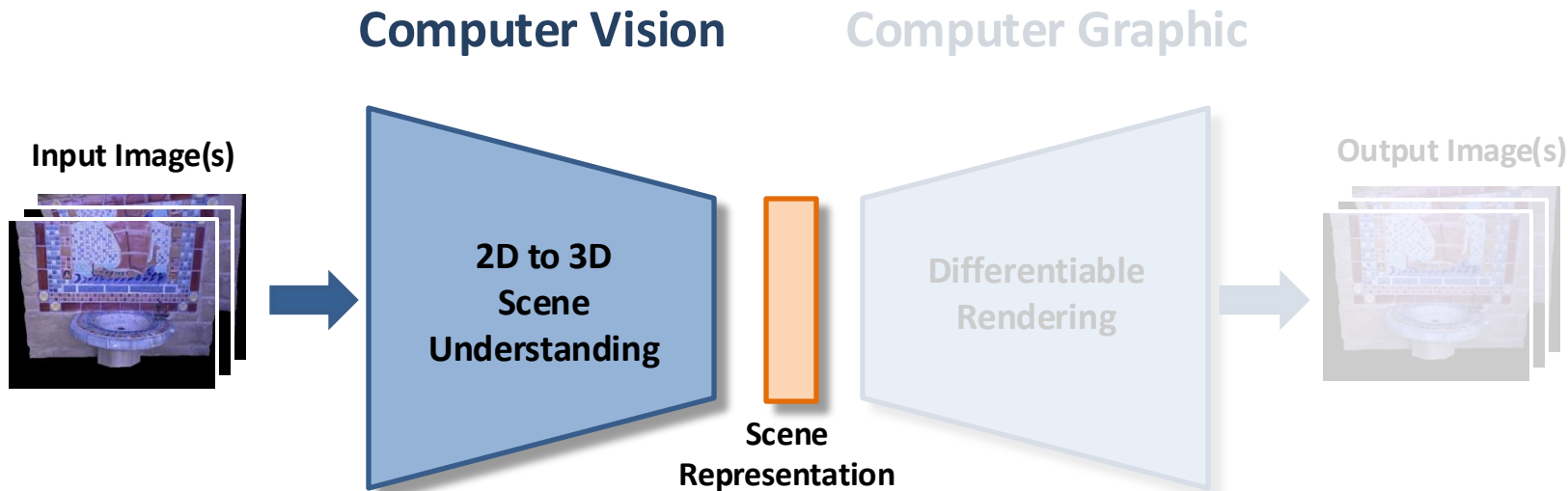


Autoencoders – Self-Supervised Learning

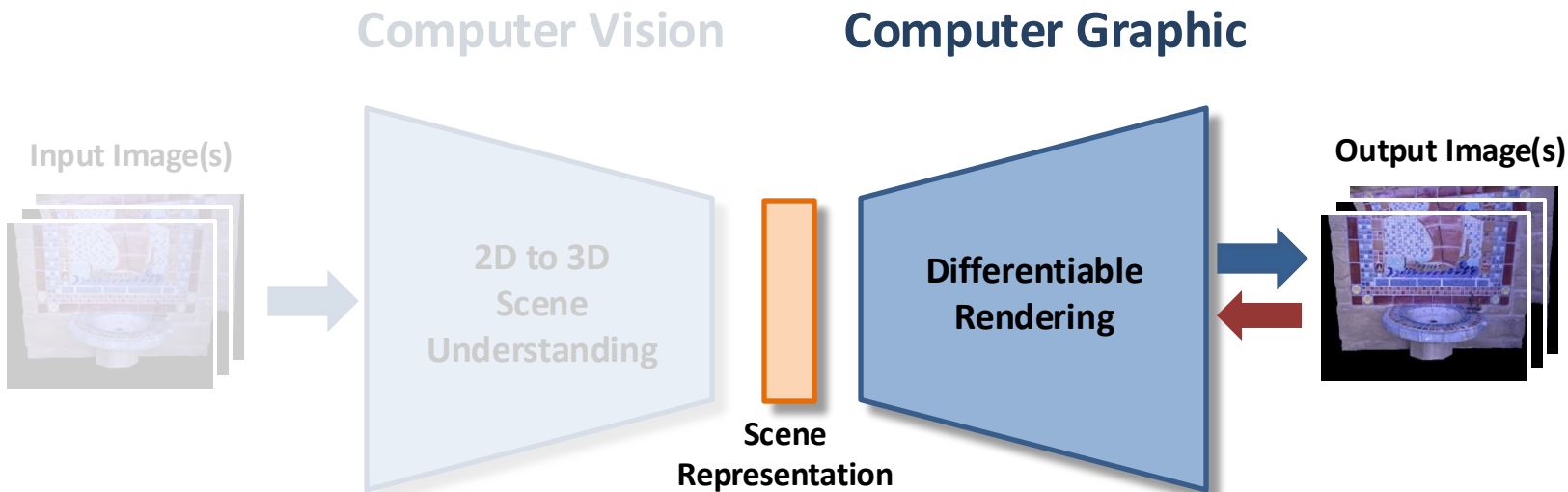


Training-Time

Autoencoders – Self-Supervised Learning

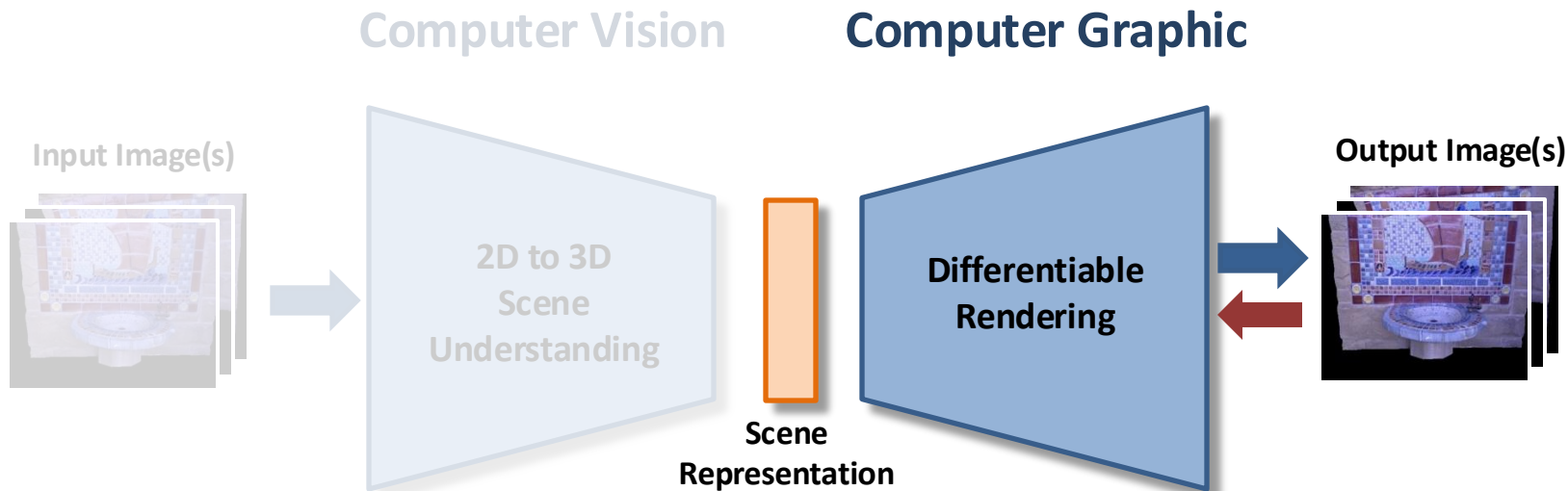


Autoencoders – Self-Supervised Learning



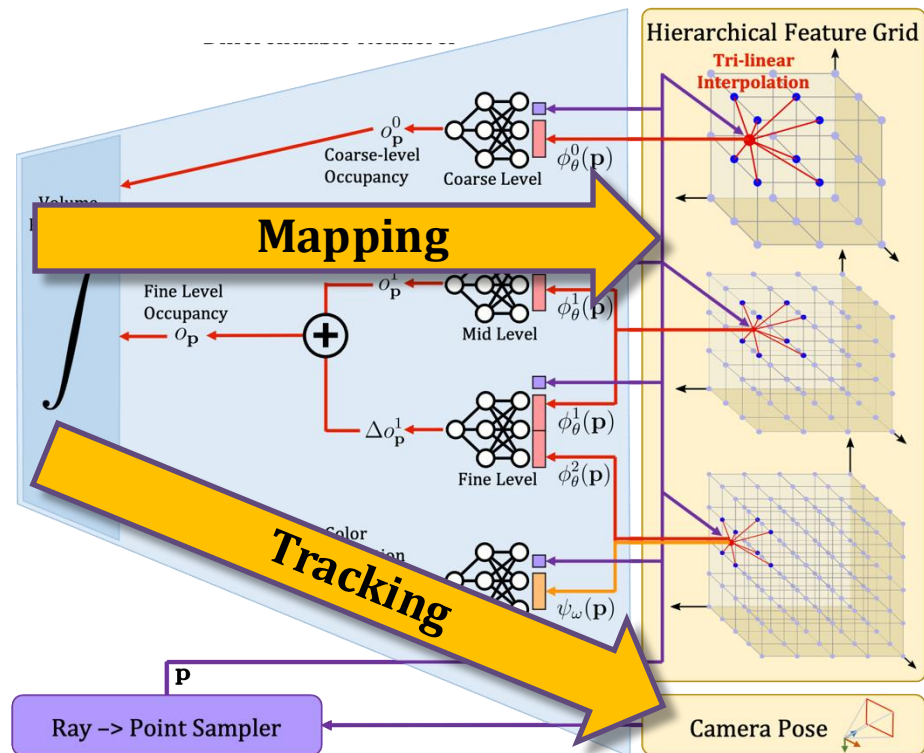
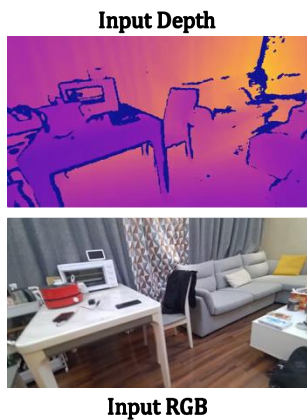
Test-Time

Autoencoders – Self-Supervised Learning



Test-Time

Example: NICE-SLAM



Questions?

Disclaimer

Many of the slides used here are obtained from online resources (including many open lecture materials) without appropriate acknowledgement. They are used here for the sole purpose of classroom teaching. All the credit and all the copyrights belong to the original authors. You should not copy it, redistribute it, put it online, or use it for any other purposes than for this course.